Engineering Applications of Artificial Intelligence 155 (2025) 110919

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

Model-based contextual reinforcement learning for robotic cooperative manipulation

Wenrui Wang [a], Qirong Tang [a,b,*], Hao Yang [a], Chao Yang [a], Baoping Ma [a], Shuqing Wang [a], Rongfu Lin [a,b]

[a] The Laboratory of Robotics and Multibody System, School of Mechanical Engineering, Tongji University, Shanghai 201804, China
[b] Shanghai Institute of Aircraft Mechanics and Control, 100 Zhangwu Road, Yangpu District, Shanghai 200092, China

ARTICLE INFO

Keywords:
Cooperative manipulation
Model-Based Contextual Reinforcement Learning
Projected inverse dynamics
Probabilistic forward model

ABSTRACT

In large-scale manufacturing and automation, cooperative robotic manipulation requires lower-level controllers to generate specific position or force commands based on real-time information. However, adapting these controllers for different objects or trajectories often demands a complete re-learning process. Contextual Reinforcement Learning (CRL) offers a method to generalize controllers across various contexts, yet it frequently faces instability and inefficiency due to uncertain contact dynamics and external disturbances, especially in real-world industrial environments. To enhance process efficiency and sustainability in robotic systems, a Model-Based Contextual Reinforcement Learning (MCRL) approach is proposed, leveraging metaheuristic solutions to optimize cooperative manipulation. The approach includes a parameterized lower-level control policy, designed using projected inverse dynamics for decoupled control in both constrained and unconstrained subspaces. A probabilistic forward model further enhances the cooperative manipulation process by calculating expected returns and mitigating risk-sensitive biases. Through these optimizations, the MCRL framework efficiently generalizes lower-level policies and improves operational performance. Evaluation on a multi-robot simulation system and a physical dual-arm platform demonstrates the approach's effectiveness in achieving high-quality, sustainable policies for cooperative manipulation tasks, aligning with the goals of improved efficiency and resource utilization in manufacturing.

1. Introduction

As manufacturing systems increasingly adopt automation and robotics to enhance efficiency, the limitations of traditional single-robot working modes become evident in handling complex manipulation tasks. Robotic cooperative manipulation, where multiple robots work in coordination, presents a promising approach to address these challenges. To optimize such systems, learning-based algorithms (Kulkarni et al., 2024; Abdelali et al., 2024), particularly reinforcement learning (RL) (Hou et al., 2020), have been widely researched due to their ability to manage dynamic and complex control tasks. RL algorithms are typically employed to learn controllers that enable robots to perform sophisticated operations. However, in cooperative manipulation involving diverse objects, uncertainties in contact dynamics and external disturbances pose significant challenges that must be addressed for successful implementation (Kupcsik et al., 2017). Effective solutions can drive not only improvements in manufacturing process efficiency but also broader sustainability outcomes, aligning with the ongoing industry emphasis on innovation and resource optimization.

1.1. Related work for robotic cooperative manipulation

Robotic cooperative manipulation has been studied somewhat extensive. Many cooperative control techniques have been developed to solve the dynamic coupling problem of robotic cooperation, among which the two most commonly used frameworks are impedance control and hybrid position/force control. The impedance controller (Xu et al., 2023) is a practical-compliant controller to regulate dynamic response of the end-effector of robotic cooperative systems. However, the performance of impedance controllers largely depends on the impedance model parameters tuning (Zhang et al., 2018). The hybrid position/force control only tracks position and force along orthogonal directions, which is not conducive to implementing complex cooperative manipulation tasks (Jiao et al., 2022). These two kinds of control framework need exact load knowledge in practical application, which may result in limited generalization ability (Culbertson et al., 2021). Some adaptive controllers and learning-based approaches are

* Corresponding author at: The Laboratory of Robotics and Multibody System, School of Mechanical Engineering, Tongji University, Shanghai 201804, China.
E-mail address: qirong.tang@outlook.com (Q. Tang).

https://doi.org/10.1016/j.engappai.2025.110919
Received 21 October 2024; Received in revised form 16 March 2025; Accepted 16 April 2025
Available online 9 May 2025
0952-1976/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

developed to address the limitation. To accommodate the parametric uncertainties, traditional adaptive approaches are used to modify controller parameters and solve the force/position tracking problem of cooperative robot manipulators (Zribi and Ahmad, 1992; Jean and Fu, 1993). Improved adaptive controller, e.g., decentralized adaptive cooperative controller (Kawasaki et al., 2006), adaptive fuzzy position/force controller (Gueaieb et al., 2003), or adaptive neural controller (Panwar et al., 2012), further generalize the ability to adapt to unknown environments to obtain better cooperation performance. Learning-based approaches, e.g., impedance learning (Kong et al., 2019) or adaptive Learning (Zhang et al., 2022), hold the promise for identifying environmental uncertainties from dynamic interactions (Hou et al., 2020). However, neither the adaptive controller nor the learning-based approaches can take into account all the environmental uncertainties, so these methods cannot achieve sufficiently flexible performance for various situations (Buşoniu et al., 2018).

The idea of reinforcement learning (RL) is that, defined only a reward function, a learning algorithm can find strategies that yield high rewards through trial and error without prior knowledge (Buchli et al., 2011). To this end, the RL technology can be considered as a direct optimal control algorithm to learn the robotic cooperative manipulation skills, instead of exerting considerable effort in establishing the specific contact models and tuning the control parameters. What is more, RL algorithm has good generalization ability to unknown uncertainties, which provides the possibility to extend the learned policies to unknown situations.

1.2. RL for robotic cooperative manipulation

The parameterized controllers are usually chosen to output actions of each robot manipulator in robotic cooperative systems, which can be learned by using RL algorithms. Policy search (PS) methods are one of the most commonly used RL algorithms for learning complex manipulation tasks (Kupcsik et al., 2017). PS methods typically are used to optimize the parameters of the parameterized controllers to obtain maximum reward. Model-free policy search algorithms update the control policy solely based parameter-reward samples without any knowledge about the real robotic cooperative system. However, high quality policies always require thousands of trials (Chatzilygeroudis et al., 2019) and it is also difficult to guarantee the system safety during the training process of robotic cooperative manipulation. Model-based policy search methods can considerably improve the data-efficiency by learning forward models of the robotic cooperative system and interaction dynamics (Jiang et al., 2023; Kroemer et al., 2021).

Model-based policy search methods are widely used in the learning of robot manipulation skills. A common model-based method to learn nonlinear models of systems is to use locally weighted Bayesian regression (LWBR) algorithm (Antonova et al., 2022; Junge et al., 2020; Dohmann et al., 2021). LWBR has been applied to learn posterior distributions of physical system to improve performance of robotic manipulation, such as deformable object manipulation (Antonova et al., 2022) and food cooking (Junge et al., 2020). Bayesian distributed learning framework was also employed in cooperative manipulation to estimate the object dynamics and overcome the system uncertainties (Dohmann et al., 2021). An alternative model-based RL method is to use Gaussian process (GP) models, which have proven to be effective in learning dynamic stochastic models (Deisenroth et al., 2013; Jang et al., 2020; Fanger et al., 2016). One of state-of-the art GP policy search methods is the probabilistic inference for learning control (PILCO) algorithm, which reduces the effects of model errors by incorporating model uncertainty into long-term planning and controller learning (Deisenroth et al., 2013). GP policy search methods like PILCO has been successfully applied for learning the controllers for multi-robot system (Jang et al., 2020) and knowledge-based cooperation (Fanger et al., 2016) with unprecedented data-efficiency.

Although model-based policy search methods have been applied for robotic cooperative control (Jang et al., 2020; Willemsen et al., 2021), hierarchical reinforcement learning (HRL) may be a more efficient scheme due to the complexity of robotic cooperative systems. In HRL framework, the contexts such as the manipulated objects or reference trajectory may change between task executions. To adapt the policy to a new context, contextual reinforcement learning (CRL) approaches are introduced to utilize HRL framework. An upper-level policy can generalize the lower-level controllers to new contexts with CRL.

1.3. CRL for robotic cooperative manipulation

Generally, the CRL works in a hierarchical framework. However, different from HRL (Hou et al., 2022), CRL does not require fix the contexts while learning robotic cooperative manipulation skills. In the CRL, the high-level abstraction of shared task structure permits simultaneous policy optimization and generalization to new contexts through interpolation and extrapolation (Kupcsik et al., 2017; Klink et al., 2020). The goal in CRL is to learn how to choose the parameters of the lower-level force/position controller as a function of the contexts. The generalized lower-level policies generate actions to achieve cooperative manipulation tasks in fixed contexts (Kupcsik et al., 2013). To this end, CRL holds the promise of driving the next wave of breakthroughs by leveraging similarities between different contexts.

Through the employment of CRL, the challenge of the reality gap can be mitigated, as human demonstrations serve to initialize the learning process and narrow down the search space (Pong et al., 2018; Abdolmaleki et al., 2016a). While working in unstructured environment, the generalization ability of CRL makes the process of cooperative manipulation skill acquisition more efficient and safe (Yu et al., 2022; Kumar et al., 2019). Within this framework, lower-level policies can be assimilated directly into new contexts through learning from human demonstrations or the interactions of the robot with manipulated objects. Specifically, CRL provides a principled framework for learning and generalization of behaviors across related tasks. Relative entropy policy search (CREPS) method, developed based on deterministic approximate inference techniques (Abdolmaleki et al., 2015) and information theoretic policy search (Peters et al., 2010), is highly suitable for broadening its application to CRL (Daniel et al., 2016). Contextual relative entropy policy search(REPS) (Abdolmaleki et al., 2016a) is developed to generalize lower-level policies for different contexts, where a context is described by a real valued vector. To address the issues of premature convergence and underperformance compared to state-of-the-art stochastic search methods in CRL, contextual relative entropy policy search with covariance matrix adaptation (CREPS-CMA) is proposed (Abdolmaleki et al., 2016b). REPS and CREPS-CMA are model-free, thereby foregoing the advantages of structured, model-based learning (Klink et al., 2020). Consequently, substantial simulations and experiments are required in the practical policy learning process. Research on model-based contextual policy search is relatively scarce; yet some achievements have established a bridge from model-based contextual reinforcement learning (MCRL) to the acquisition of robotic manipulation skills. This has been achieved through the acquisition of local quadratic reward models concerning policy parameters and contextual variables (Jin et al., 2024), or through the acquisition of probabilistic forward models of the system (Kupcsik et al., 2017), enabling robots to perform tasks such as ball-throwing, robot hockey, and table tennis. However, owing to the intricacies of designing lower-level policies and the uncertainties inherent in system dynamics, MCRL is rarely employed in robotic cooperative manipulation tasks. This constraint somewhat impedes the generalization potential of RL in the field of robotic cooperative manipulation.

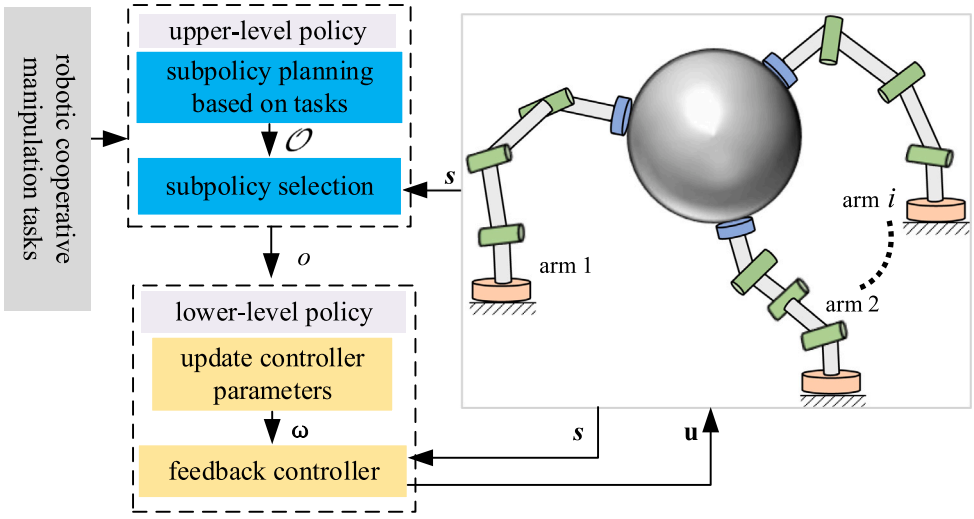


Fig. 1. Schematic diagram of a HRL framework for robotic cooperative manipulation.

1.4. Contributions

In this paper, MCRL is innovatively applied to the robotic cooperative manipulation. This study entails acquiring probabilistic forward models for robotic cooperative manipulation systems and employing projected inverse dynamics to design parameterized decoupled control policies at the lower level. The learned forward models are utilized to generate artificial experiment outcomes, which are subsequently employed for the upper-level policy updates (Tangkaratt et al., 2017). Projected inverse dynamics allow the decomposition of lower-level control into constrained and unconstrained subspaces (Lin et al., 2018). The desired motion is achieved within the unconstrained subspace, while contact and friction constraints are enforced within the constrained subspace. The learned forward models are used to generate lower policy parameters in constrained and unconstrained subspaces. The main contributions are summarized as follows:

(1) The introduction of MCRL innovatively promotes the efficient and high-quality generalization of lower-level robot control policies to new contexts through the learning of upper-level policies. This method design enables robotic cooperation systems to acquire cooperative skills with significantly fewer trials.

(2) The parameterized decoupled control policy based on projected inverse dynamics is firstly employed in CRL framework. They serve as the lower-level policy tasked with searching for policy parameters to generate desired motion and ensure constraint satisfaction within the proposed MCRL algorithms.

(3) A Gaussian process model is employed to fit the forward model of robotic cooperative manipulation, generating simulation and artificial experimental outcomes for subsequent policy updates. Reusing high-performing datasets to update the model mitigates the influence of the complexity and uncertainty of system dynamics, thereby ensuring the quality and efficiency of policy acquisition.

2. Parameterized decoupled lower-level policies

Robotic cooperative manipulation, as depicted in Fig. 1, is effectively addressed within the hierarchical reinforcement learning (HRL) framework (Kupcsik et al., 2017). In this section, parameterized decoupled lower-level policies are introduced to adapt to diverse contexts by adjusting their parameters. The lower-level controllers aim to achieve the desired motion of manipulated object while ensuring compliance with cooperative grasping constraints. Due to the complexity and uncertainty of system dynamics, designing and simplifying lower-level controllers pose significant challenges. Projected inverse dynamics is utilized to decouple the control of the cooperative system into unconstrained and constrained subspaces. Different from Lin et al. (2018), Dehio et al. (2018), the effects of constrained and unconstrained wrenches are analyzed directly at the point of contact. Since the wrenches on the contact point can be easily obtained by a force sensor equipped, it is more convenient for subsequent adjustment and optimization. In this way, the unconstrained wrenches propel the manipulated objects to achieve the desired motion, whereas the constrained wrenches uphold the grasp constraints between the robotic arm's end-effector and the manipulated object.

2.1. Projected inverse dynamics for robotic cooperative manipulation system

For multi-arm robotic manipulators grasping and manipulating a rigid object via a force-closed chain constraint, all the end-effectors are in contact with the object, and they can generate arbitrary wrenches (force/torque combinations) on the object. The generated wrenches not only enforce the force-closed grasp of the manipulated object, but also drive the object to achieve the desired motion.

The wrenches upon the manipulated object are illustrated in Fig. 2. The mass of the manipulated object is denoted by m. $\boldsymbol{g}$ represents the gravitational acceleration vector. $\boldsymbol{F}_{dis}$ is assigned as the disturbing wrenches, while $\boldsymbol{f}_1$, $\boldsymbol{f}_2$, and $\boldsymbol{f}_i$ denote the wrenches applied to the manipulated object by the first, second, and ith manipulators, respectively. According to the projected inverse dynamics (Lin et al., 2018; Mistry and Righetti, 2012), these wrenches can be decomposed into constrained and unconstrained components

$$\boldsymbol{f} = [\boldsymbol{f}_1, \boldsymbol{f}_2, \ldots, \boldsymbol{f}_i]^T = \boldsymbol{Q}\boldsymbol{f} + (\boldsymbol{I} - \boldsymbol{Q})\boldsymbol{f}, \tag{1}$$

where $\boldsymbol{Q} = \boldsymbol{G}^{+}\boldsymbol{G}$ is the orthogonal projection matrix that can project $\boldsymbol{f}$ onto the constrained subspace. To ensure clarity, the term "wrench" is used throughout the manuscript to specifically refer to force/torque combinations, distinguishing it from pure force. Intuitively, the projection matrix $\boldsymbol{Q}$ serves as a transformation operator that extracts the component of the applied wrenches that contributes to maintaining the grasp constraints, while the remaining portion is mapped to the unconstrained subspace to drive object motion. $\boldsymbol{G}$ is the grasp matrix of the robotic cooperative manipulation system and $\boldsymbol{G}^{+} = \boldsymbol{G}^{T}\left(\boldsymbol{G}\boldsymbol{G}^{T}\right)^{-1}$ is the Pseudo-inverse matrix of the grasp matrix $\boldsymbol{G}$. This decomposition allows for independent control of the constrained and unconstrained wrenches, ensuring stability in grasping while achieving precise object

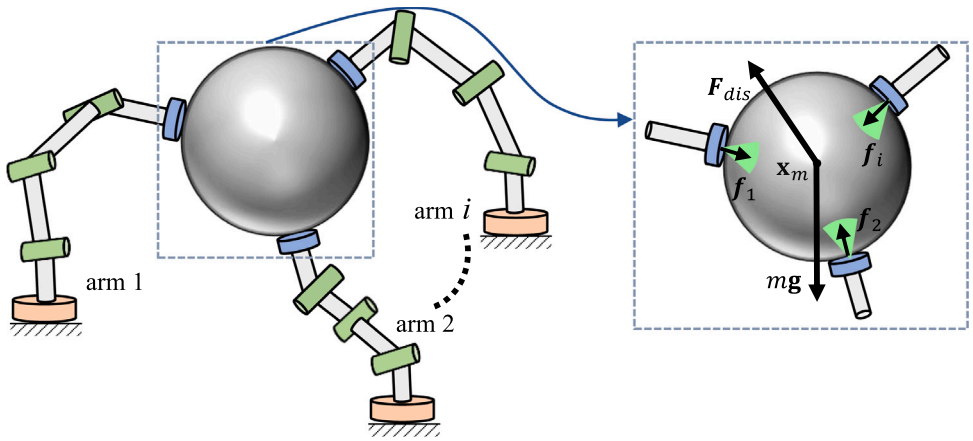


Fig. 2. Illustration of wrenches in a multi-arm manipulation.

motion. For cooperative manipulation system with i robotic manipulators, the grasp map $\boldsymbol{G}$ is the horizontal concatenation of i grasp matrices

$$\boldsymbol{G} = [\boldsymbol{G}_1, \boldsymbol{G}_2, \ldots, \boldsymbol{G}_i] \in \mathbb{R}^{6\times 6i}. \tag{2}$$

The grasp matrix for a single robotic arm delineates the relationship between the manipulated object's twist and the contact twists (Murray et al., 2017).

$$\boldsymbol{G}_i = \begin{bmatrix} \boldsymbol{R}_i & \boldsymbol{0} \\ \mathbb{S}(\boldsymbol{d}_i) & \boldsymbol{R}_i \end{bmatrix}, \tag{3}$$

where $\boldsymbol{R}_i \in \mathbb{R}^{3\times 3}$ is the rotation matrix between the ith contact frame and the manipulation object frame, $\mathbb{S}(\boldsymbol{d}_i)$ is defined as

$$\mathbb{S}(\boldsymbol{d}_i) = \begin{bmatrix} 0 & -d_z & d_y \\ d_z & 0 & -d_x \\ -d_y & d_x & 0 \end{bmatrix}, \tag{4}$$

where d_x, d_y and d_z are the distances along the axes between the ith contact position to the object center-of-mass.

The constrained wrench vector $\boldsymbol{f}^C = \boldsymbol{Q}\boldsymbol{f}$ and the unconstrained wrench vector $\boldsymbol{f}^U = (\boldsymbol{I} - \boldsymbol{Q})\boldsymbol{f}$ are orthogonal described in (1). $\boldsymbol{f}^C, \boldsymbol{f}^U \in \mathbb{R}^{6\times i}$ can be calculated with

$$\boldsymbol{f}^C = [\boldsymbol{f}_1^C, \boldsymbol{f}_2^C, \ldots, \boldsymbol{f}_i^C]^T = \boldsymbol{G}^+\boldsymbol{G}[\boldsymbol{f}_1, \boldsymbol{f}_2, \ldots, \boldsymbol{f}_i]^T, \tag{5}$$

$$\boldsymbol{f}^U = [\boldsymbol{f}_1^U, \boldsymbol{f}_2^U, \ldots, \boldsymbol{f}_i^U]^T = (\mathbf{I} - \boldsymbol{G}^+\boldsymbol{G})[\boldsymbol{f}_1, \boldsymbol{f}_2, \ldots, \boldsymbol{f}_i]^T, \tag{6}$$

where $\boldsymbol{G}^+ = \boldsymbol{G}^T(\boldsymbol{G}\boldsymbol{G}^T)^{-1}$ is the Pseudo-inverse matrix of the grasp matrix $\boldsymbol{G}$. As $\boldsymbol{f}^C$ acting in the null space of grasp map should not produce any virtual work for any virtual displacement, the cooperative manipulation system is constrained such that only $\boldsymbol{f}^C$ is allowed to enforce the contacts. Meanwhile, only $\boldsymbol{f}^U$ drives the manipulated object to achieve the desired motion (Erhart and Hirche, 2015), and the unconstrained space control law does not involve the constraint wrenches. This decomposition enables independent control of the constrained and unconstrained wrenches, allowing for the design of dedicated controllers based on different control objectives.

2.2. Grasp constraints an analysis of robotic cooperative manipulation system

Due to the orthogonal relationship between the unconstrained and constrained wrenches, $\boldsymbol{f}^U$ does not contribute to maintaining grasp constraints. Adjustments to the constrained wrenches will also not affect the motion of the manipulated object. Therefore, the analysis primarily focuses on the role of constrained wrenches in preserving grasp constraints. Specifically, the individual components $\boldsymbol{f}_i^C$ can be expressed as

$$\boldsymbol{f}_i^C = [f_{i,x}^C, f_{i,y}^C, f_{i,z}^C, t_{i,x}^C, t_{i,y}^C, t_{i,z}^C]^T. \tag{7}$$

$f_{i,x}^C$, $f_{i,y}^C$ and $f_{i,z}^C$ are the forces acting by the ith manipulator on the manipulated object along the direction of x, y and z, respectively; $t_{i,x}^C$, $t_{i,y}^C$ and $t_{i,z}^C$ are the moment along each axis. Here, z-axis is along the direction normal to the contact surface. According to Fig. 2, the grasp constraints are defined as follows

(1) Contact constraints: In the case of Fig. 2, to maintain contacts between the manipulators and the manipulated object, the manipulators should only push toward the contact, but not pull. Meanwhile, to prevent damage to the manipulation object, the squeezing forces should not be excessive. Hence, the contact normal should satisfy the contact constraints

$$0 < f_{i,z}^C < f_{dam}, \tag{8}$$

where f_{dam} is a threshold, if $f_{i,z}^C$ is larger than it, the manipulated object may be damaged, but f_{dam} cannot be obtained by direct observation but only by actual testing.

(2) Friction Constraints: To prevent relative slipping between the manipulators and the manipulated object, the tangential wrenches should not exceed the friction wrenches. The Coulomb friction model is usually used to describe the contact (Righetti et al., 2013). By Coulomb's Law, the contact wrench $\boldsymbol{f}_i$ should satisfy

$$\mu^f f_{i,z}^C \geq \sqrt{f_{i,x}{}^2 + f_{i,y}{}^2}, \tag{9}$$

where μ^f is the friction coefficient which depends on the contact materials between the manipulators and object. Geometrically, the applied wrenches are considered to lie in a green cone (shown in Fig. 2) centered about the direction normal to the contact surface (Trinkle et al., 1997).

(3) Moment Constraints: To avoid the manipulators from rolling at the contact point, the surface friction should be large enough to generate friction force, furthermore, the constraints are imposed on the torsional moment

$$\mu^t f_{i,z}^C \geq |t_{i,z}|, \tag{10}$$

where μ^t is the torsional friction coefficient. It is assumed that the center of pressure of the contact remains within the hand's contact patch.

2.3. Parameterized decoupled lower-level control policy

This section primarily addresses the control of end-effector contact wrenches in robotic cooperative manipulation processes, which are

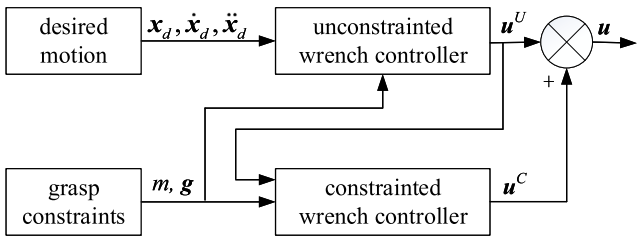


Fig. 3. An overview of the constrained and constrained wrench hybrid controller.

essential for successful task completion. A hybrid constrained and unconstrained wrench decoupled control framework is developed as the lower-level policy. The selection of controller parameters is guided by the learned Contextual Reinforcement Learning (CRL) policy, ensuring robustness within specific contexts. Control inputs are generated by the parameterized controller to fulfill grasping constraints while driving the manipulated object to achieve desired motion.

Considering that unconstrained wrenches propel the motion of the manipulated object, converting work into kinetic and potential energy, the dynamic equation for the unconstrained wrench vector $\boldsymbol{f}^U$ can be established as

$$m\ddot{\boldsymbol{x}} + m\boldsymbol{g} + \boldsymbol{F}_{dis} = \boldsymbol{G}\boldsymbol{f}^U. \tag{11}$$

Drawing from inverse dynamics control principles (Spong et al., 2020), the control input for the unconstrainted wrench vector can be delineated as.

$$\boldsymbol{u}^U = \boldsymbol{G}^+ \left(m\ddot{\boldsymbol{x}}_{\boldsymbol{a}} + m\boldsymbol{g} + \boldsymbol{F}_{dis}\right), \tag{12}$$

where the term $\ddot{\boldsymbol{x}}_{\boldsymbol{a}}$ represents a new input to the system which is yet to be chosen. It is obvious that $\ddot{\boldsymbol{x}}_{\boldsymbol{a}} = \ddot{\boldsymbol{x}}$ by combining (11) and (12), so each input $\ddot{\boldsymbol{x}}_{\boldsymbol{a}}$ can be designed to control a scalar linear system. The obvious choice of $\ddot{\boldsymbol{x}}_{\boldsymbol{a}}$ is to set

$$\ddot{\boldsymbol{x}}_{\boldsymbol{a}} = \ddot{\boldsymbol{x}}_{\boldsymbol{d}} + \boldsymbol{K}_{\boldsymbol{P}}\boldsymbol{e} + \boldsymbol{K}_{\boldsymbol{D}}\dot{\boldsymbol{e}}, \tag{13}$$

where $\boldsymbol{K}_{\boldsymbol{P}} = diag(k_{p1}, \ldots, k_{p6})$ and $\boldsymbol{K}_{\boldsymbol{D}} = diag(k_{d1}, \ldots, k_{d6})$ are diagonal matrices with diagonal elements consisting of pose and velocity gains, respectively; $\ddot{\boldsymbol{x}}_{\boldsymbol{d}}$ is the desired acceleration of the manipulated object and is used as a feed-forward input in the controller; $\boldsymbol{e}$ and $\dot{\boldsymbol{e}}$ are respectively the pose deviation and velocity deviation. Furthermore, the input unconstrainted wrench vector is solved with using (12) and (13)

$$\boldsymbol{u}^U = \boldsymbol{G}^+ \left(m(\ddot{\boldsymbol{x}}_{\boldsymbol{d}} + \boldsymbol{K}_{\boldsymbol{P}}\boldsymbol{e} + \boldsymbol{K}_{\boldsymbol{D}}\dot{\boldsymbol{e}}) + m\boldsymbol{g} + \boldsymbol{F}_{dis}\right). \tag{14}$$

The effect of constrainted wrenches is to maintain the contact constraints, friction constraints and moment constraints. To prevent relative slipping or rolling between the end-effector and manipulated object, the constrainted wrenches should increase with the wrenches to make the friction greater than the tangential wrenches. Because only the constrainted wrenches squeeze (or push) the object and create the corresponding friction wrenches, the input constrainted wrench vector can be defined as

$$\boldsymbol{u}^C = \boldsymbol{f}^C_{basis} + \boldsymbol{f}^C_{con}, \tag{15}$$

where $\boldsymbol{f}^C_{basis}$ represents the fixed term that keeps the contact constraints, $\boldsymbol{f}^C_{con}$ is the adjustable term that maintains the friction and moment constraints. $\boldsymbol{f}^C_{basis}$ is mainly adjusted according to the motion of the manipulated object. It is not hard to find from (9) and (10) that sliding friction and torsional friction are generally related to the normal component to the contact surface. In this way, the constraint wrench along the z axis is defined as

$$f^C_{i,z} = \begin{cases} f^{up}_{i,z} & k_{\text{int}} f_{i,ta} \geq f^{up}_z \\ k_{\text{int}} f_{i,ta} & f^{up}_z \geq k_{\text{int}} f_{i,ta} \geq f^{down}_z \\ f^{down}_{i,z} & k_{\text{int}} f_{i,ta} \leq f^{down}_z \end{cases}, \tag{16}$$

where $f^{up}_{i,z}$ and $f^{down}_{i,z}$ represent the upper and lower bounds of constraint wrenches along the z direction respectively, k_{int} is the corresponding proportional gain coefficient, $f_{i,ta}$ is selected by $f_{i,ta} = \max[\sqrt{f_{i,x}{}^2 + f_{i,y}{}^2}, |t_{i,z}|]$. Meanwhile, the components in all directions except the z axis are defined as 0. Furthermore, as illustrated in Fig. 3, the input $\boldsymbol{u} \in \mathbb{R}^{6\times i}$ can be written as

$$\boldsymbol{u} = \boldsymbol{u}^U + \boldsymbol{u}^C, \tag{17}$$

where $\boldsymbol{u}$ is contact wrench vector between i cooperative robotic arm and manipulated object, and yields

$$\boldsymbol{u} = [\boldsymbol{f}_1, \boldsymbol{f}_2, \ldots, \boldsymbol{f}_i]^T. \tag{18}$$

The lower-level controller can be parameterized as

$$\boldsymbol{u} = \pi^L(\boldsymbol{s}; \boldsymbol{\omega}), \tag{19}$$

where π^L represents the parameterized decoupled lower-level policy, $\boldsymbol{s}$ is the state of manipulated object, including displacement and wrench components, $\boldsymbol{\omega} \in \Omega$ is the parameterized lower-level policy vector, and Ω is the space of lower-level policy parameters. $\boldsymbol{\omega}$ can be written as

$$\boldsymbol{\omega} = [m, \boldsymbol{k}_P, \boldsymbol{k}_D, \boldsymbol{g}, \boldsymbol{F}_{dis}, \boldsymbol{f}^C_{basis}, k_{\text{int}}], \tag{20}$$

where $\boldsymbol{k}_P = [k_{p1}, \ldots, k_{p6}]$ and $\boldsymbol{k}_D = [k_{d1}, \ldots, k_{d6}]$ are matrices reconstructed by extracting the diagonal elements of $\boldsymbol{K}_P$ and $\boldsymbol{K}_D$, respectively. In practice, to reduce the complexity and maintain the stable behaviors, only $\boldsymbol{k}_P$, $\boldsymbol{k}_D$, $\boldsymbol{f}^C_{basis}$, and k_{int} are selected as the lower-level policy parameter $\boldsymbol{\omega}$ to learn. Hence, $\boldsymbol{\omega}$ can be rewritten as

$$\boldsymbol{\omega} = [\boldsymbol{k}_P, \boldsymbol{k}_D, \boldsymbol{f}^C_{basis}, k_{\text{int}}]. \tag{21}$$

In the framework of such parameterized lower-level policies, the selection and updating of parameters are primarily influenced by the mass of the manipulated object and its desired trajectory, while disregarding the effects of the object's shape and surface roughness. The parameterized lower-level policy facilitates the learning of controller parameters across various contexts. However, obtaining a new set of controller parameters for each context entails significant training and computational expenses, making it impractical for real-world applications. Thus, the development of a contextual reinforcement learning (CRL) method is crucial to optimize the hierarchical framework and enhance generalization capabilities.

3. Model-based contextual reinforcement learning

Generally, contextual reinforcement learning (CRL) works within the framework of hierarchical approaches, which are utilized to learn a higher-level policy that generalizes lower-level controllers to new contexts. Based on standard Markov decision processes (MDPs), hierarchical policies can be introduced to enhance the efficiency of solving robotic cooperative manipulation problems, as illustrated in Fig. 1. Hierarchical policy can be expressed as

$$\pi(\boldsymbol{a}|\boldsymbol{s}) = \sum_{o\in\mathcal{O}} \pi(o|\boldsymbol{s})\,\pi(\boldsymbol{a}|\boldsymbol{s}, o). \tag{22}$$

According to the policy $\pi(\boldsymbol{a}|\boldsymbol{s})$, the agent chooses a corresponding action $\boldsymbol{a} \in \mathcal{A}$ when the current state is $\boldsymbol{s} \in S$. In this framework, a sub-policy is firstly determined from the gating network $\pi(o|\boldsymbol{s})$ and, subsequently, the action $\boldsymbol{a}$ is chosen from the specific sub-policy $\pi(\boldsymbol{a}|\boldsymbol{s}, o)$. Sub-policy set $\mathcal{O}$ covers all possible options o. In this scenario, all options pertain to the mass and desired motion of the manipulated object. The practical feasibility of applications is ensured by considering the acceleration and mass of the manipulated object as context variables, rather than introducing higher-dimensional context variables (Tangkaratt et al., 2017). Therefore, context variables are directly determined by designers at the planning level or obtained through simple measurements, eliminating the need for higher-level strategy exploration.

The average reward with the state–action distribution is denoted as follows:

$$R(\pi) = \int\int \mu^{\pi}(s) \sum_{o\in\mathcal{O}} \pi(o|s)\,\pi(\boldsymbol{a}|s,o)\, r(s,\boldsymbol{a})\, ds d\boldsymbol{a}, \tag{23}$$

where $r(s,\boldsymbol{a})$ is a reward that the agent receives after executing the action $\boldsymbol{a}$ in state s, $\mu^{\pi}(s)$ is the state visit distribution of policy π.

3.1. CRL

The context vector $\boldsymbol{o}$ for the robotic cooperative manipulation problem is defined as the set of variables that fully specify the option o. Different parameter vectors $\boldsymbol{\omega}$ need to be chosen in each context. Considering the episode-based policy search framework (Kupcsik et al., 2017; Song and Scaramuzza, 2022), the wrench profile is defined as the set of state–action pairs at each time step of the episode of length T

$$\boldsymbol{\tau} = \left\{s_1, \boldsymbol{u}_1, \ldots, s_t, \boldsymbol{u}_t, \ldots s_T, \boldsymbol{u}_T\right\}, \tag{24}$$

where the output $\boldsymbol{u}_t$ of the lower-level policy is considered to be an action $\boldsymbol{a}_t$ at time step t. In this CRL framework, the policy is represented by a search distribution over $\pi(\boldsymbol{\omega})$ and Ω, it is usually defined as a Gaussian distribution

$$\pi(\boldsymbol{\omega}) = \mathrm{N}\left(\boldsymbol{\omega}\,|\mu_{\boldsymbol{\omega}}, \Sigma_{\boldsymbol{\omega}}\right), \tag{25}$$

where $\mu_{\boldsymbol{\omega}}$ and $\Sigma_{\boldsymbol{\omega}}$ are the mean and the variance of $\boldsymbol{\omega}$ respectively. Furthermore, the optimal policy is obtained by solving

$$\pi^*(\boldsymbol{\omega}) = \arg\max_{\pi} \int_{\Omega} \pi(\boldsymbol{\omega})\, R_{\boldsymbol{\omega}} d_{\boldsymbol{\omega}}, \tag{26}$$

where $R_{\boldsymbol{\omega}}$ is the expected episode reward, it can be denoted as

$$R_{\boldsymbol{\omega}} = \int_{\tau} p(\boldsymbol{\tau}|\boldsymbol{\omega})\, R(\boldsymbol{\tau}) d\boldsymbol{\tau}, \tag{27}$$

where $R(\boldsymbol{\tau})$ represents the reward of the corresponding state–action set executed, it may be defined as the sum of immediate rewards $r(s_t, \boldsymbol{u}_t)$. $p(\boldsymbol{\tau}|\boldsymbol{\omega})$ is the probability of the wrench profile given the parameters $\boldsymbol{\omega}$ of the low-level controller.

The objective of contextual policy search is to generate multiple parameterized decoupled lower-level controllers $\boldsymbol{u} = \pi^L(s;\boldsymbol{\omega})$ corresponding to different contexts. It is assumed that the context variable is fully observable, and is drawn from an unknown distribution $\boldsymbol{o} \sim \mu(\boldsymbol{o})$. The goal of learning is to find the optimal policy to maximize the expected reward

$$\pi^*(\boldsymbol{\omega}|\boldsymbol{o}) = \arg\max_{\pi} \int_{o} \mu(\boldsymbol{o}) \int_{\Omega} \pi(\boldsymbol{\omega}|\boldsymbol{o}) R_{\boldsymbol{o\omega}} d_{\boldsymbol{\omega}} d\boldsymbol{o}, \tag{28}$$

where $R_{\boldsymbol{o\omega}}$ is the expected reward when executing the lower-level policy with parameter $\boldsymbol{\omega}$ in context o. Due to the stochasticity of robot dynamics and contact environment dynamics, the reward $R_{\boldsymbol{o\omega}}$ is given by the expected reward over all wrench profiles

$$R_{\boldsymbol{o\omega}} = E_{\boldsymbol{\tau}}\left[R(\boldsymbol{\tau},\boldsymbol{o})\,|\boldsymbol{o},\boldsymbol{\omega}\right] = \int_{\boldsymbol{\tau}} p(\boldsymbol{\tau}|\boldsymbol{o},\boldsymbol{\omega})\, R(\boldsymbol{\tau},\boldsymbol{o})\, d\boldsymbol{\tau}. \tag{29}$$

Both the probability $p(\boldsymbol{\tau}|\boldsymbol{o},\boldsymbol{\omega})$ of a wrench profile and the reward function $R(\boldsymbol{\tau},\boldsymbol{o})$ depends on the context $\boldsymbol{o}$.

The CRL method is developed based on the Relative Entropy Policy Search (REPS) algorithm (Peters et al., 2010; Daniel et al., 2016). REPS is an information theoretic policy search approach, and its intuition is to maximize the expected reward while staying close to the observed data. The constraint of exploration and experience loss is implemented by bounding the relative entropy, which is also called Kullback–Leibler (KL) divergence. The KL-bound is used to define the exploration–exploitation tradeoff intuitively. In the episodic RL setting, the context vector $\boldsymbol{o}$ and the parameter $\boldsymbol{\omega}$ uniquely determine the wrench profile distribution. To this end, the distribution can be abstracted as the joint distribution over the parameter vector $\boldsymbol{\omega}$ and the context $\boldsymbol{o}$, i.e $p(\boldsymbol{o},\boldsymbol{\omega}) = \mu(\boldsymbol{o})\,\pi(\boldsymbol{\omega}|\boldsymbol{o})$ (Peters et al., 2010). Moreover, the KL-bound of REPS method is defined as

$$\int_{\boldsymbol{o},\boldsymbol{\omega}} p(\boldsymbol{o},\boldsymbol{\omega}) \log \frac{p(\boldsymbol{o},\boldsymbol{\omega})}{q(\boldsymbol{o},\boldsymbol{\omega})} d\boldsymbol{o} d\boldsymbol{\omega} \le \varepsilon, \tag{30}$$

where $p(\boldsymbol{o},\boldsymbol{\omega})$ represents the updated context-parameter distribution; $q(\boldsymbol{o},\boldsymbol{\omega})$ is the previously used distribution; $\varepsilon > 0$ is the upper bound of the KL-bound.

The context distribution $\mu(\boldsymbol{o})$ cannot be chosen freely, and it needs also satisfy the constraint $\int_{\boldsymbol{\omega}} p(\boldsymbol{o},\boldsymbol{\omega}) = \mu(\boldsymbol{o})$ for any contexts. In our work, this requirement would result in an infinite number of constraints, so only feature averages need to be matched (Daniel et al., 2016)

$$\int_{\boldsymbol{o}}\int_{\boldsymbol{\omega}} p(\boldsymbol{o},\boldsymbol{\omega})\boldsymbol{\varphi}(\boldsymbol{o}) d\boldsymbol{\omega} d\boldsymbol{o} = \hat{\boldsymbol{\varphi}}_0, \tag{31}$$

where $\hat{\boldsymbol{\varphi}}_0$ is the average observation feature vector of the initial state, and $\boldsymbol{\varphi}(\boldsymbol{o})$ is the feature vector corresponding to the context $\boldsymbol{o}$. The REPS-based contextual learning problem is now defined as

$$\begin{aligned} &\max_{p} \iint_{\boldsymbol{o},\boldsymbol{\omega}} p(\boldsymbol{o},\boldsymbol{\omega})\, R_{\boldsymbol{o\omega}} d\boldsymbol{o} d\boldsymbol{\omega} \\ &\text{s.t.} \iint_{\boldsymbol{o},\boldsymbol{\omega}} p(\boldsymbol{o},\boldsymbol{\omega}) \log \frac{p(\boldsymbol{o},\boldsymbol{\omega})}{q(\boldsymbol{o},\boldsymbol{\omega})} d\boldsymbol{o} d\boldsymbol{\omega} \leqslant \varepsilon \\ &\iint_{\boldsymbol{o},\boldsymbol{\omega}} p(\boldsymbol{o},\boldsymbol{\omega})\boldsymbol{\varphi}(\boldsymbol{o})\, d\boldsymbol{\omega} d\boldsymbol{o} = \hat{\boldsymbol{\varphi}}_0 \\ &\iint_{\boldsymbol{o},\boldsymbol{\omega}} p(\boldsymbol{o},\boldsymbol{\omega}) d\boldsymbol{\omega} d\boldsymbol{o} = 1 \end{aligned}. \tag{32}$$

The constrained optimization problem of (32) can be solved by the Lagrange multiplier method

$$\begin{aligned} L = &\iint_{\boldsymbol{o},\boldsymbol{\omega}} p(\boldsymbol{o},\boldsymbol{\omega})\, R_{\boldsymbol{o\omega}} d\boldsymbol{o} d\boldsymbol{\omega} \\ &+ \eta_L \left(\varepsilon - \iint_{\boldsymbol{o},\boldsymbol{\omega}} p(\boldsymbol{o},\boldsymbol{\omega}) \log \frac{p(\boldsymbol{o},\boldsymbol{\omega})}{q(\boldsymbol{o},\boldsymbol{\omega})} d\boldsymbol{o} d\boldsymbol{\omega}\right) \\ &+ \boldsymbol{\theta}_L^T \left(\hat{\boldsymbol{\varphi}}_0 - \iint_{\boldsymbol{o},\boldsymbol{\omega}} p(\boldsymbol{o},\boldsymbol{\omega})\boldsymbol{\varphi}(\boldsymbol{o})\, d\boldsymbol{\omega} d\boldsymbol{o}\right) \\ &+ \lambda_L \left(1 - \iint_{\boldsymbol{o},\boldsymbol{\omega}} p(\boldsymbol{o},\boldsymbol{\omega}) d\boldsymbol{\omega} d\boldsymbol{o}\right) \end{aligned}, \tag{33}$$

where η_L, $\boldsymbol{\theta}_L$ and λ_L are the Lagrangian parameters. The partial derivative of the probability distribution $p(\boldsymbol{o},\boldsymbol{\omega})$ with respect to (33) can be obtained

$$\partial_{p(\boldsymbol{o},\boldsymbol{\omega})} L = R_{\boldsymbol{o\omega}} - \eta_L \log \frac{p(\boldsymbol{o},\boldsymbol{\omega})}{q(\boldsymbol{o},\boldsymbol{\omega})} + \eta_L - \boldsymbol{\theta}_L^T \boldsymbol{\varphi}(\boldsymbol{o}) - \lambda_L. \tag{34}$$

By setting $\partial_{p(\boldsymbol{o},\boldsymbol{\omega})} L$ to zero, $p(\boldsymbol{o},\boldsymbol{\omega})$ can be denoted as

$$p(\boldsymbol{o},\boldsymbol{\omega}) = q(\boldsymbol{o},\boldsymbol{\omega}) \exp(V(\boldsymbol{o})) \exp\left(1 - \frac{\lambda_L}{\eta_L}\right), \tag{35}$$

with $V(\boldsymbol{o}) = \left(\frac{R_{\boldsymbol{o\omega}} - \boldsymbol{\theta}_L^T \boldsymbol{\varphi}(\boldsymbol{o})}{\eta_L}\right)$. The distribution can be optimized using the Expectation Maximization algorithm (Peters et al., 2010). Due to the constraint $\iint_{\boldsymbol{o},\boldsymbol{\omega}} p(\boldsymbol{o},\boldsymbol{\omega}) d\boldsymbol{\omega} d\boldsymbol{o} = 1$, it can be obtained

$$\exp\left(1 - \frac{\lambda_L}{\eta_L}\right) = \left[\iint_{\boldsymbol{o},\boldsymbol{\omega}} q(\boldsymbol{o},\boldsymbol{\omega}) \exp(V(\boldsymbol{o})) d\boldsymbol{\omega} d\boldsymbol{o}\right]^{-1}. \tag{36}$$

By reintroducing the solution of $p(\boldsymbol{o},\boldsymbol{\omega})$ into the Lagrangian, the dual function is obtained

$$\begin{aligned} g\left(\eta_L, \boldsymbol{\theta}_L, \lambda_L\right) &= \eta_L \varepsilon - \eta_L + \lambda_L + \boldsymbol{\theta}_L^T \hat{\boldsymbol{\varphi}}_0 \\ &= -\eta_L \exp\left(1 - \frac{\lambda_L}{\eta_L}\right) + \eta_L \varepsilon + \boldsymbol{\theta}_L^T \hat{\boldsymbol{\varphi}}_0 \end{aligned}. \tag{37}$$

Setting (36) to (37),

$$\begin{aligned} g\left(\eta_L, \boldsymbol{\theta}_L\right) = &\eta_L \log\left(\iint_{\boldsymbol{o},\boldsymbol{\omega}} q(\boldsymbol{o},\boldsymbol{\omega}) \exp V(\boldsymbol{o}) d\boldsymbol{\omega} d\boldsymbol{o}\right) \\ &+ \eta_L \varepsilon + \boldsymbol{\theta}_L^T \hat{\boldsymbol{\varphi}}_0. \end{aligned} \tag{38}$$

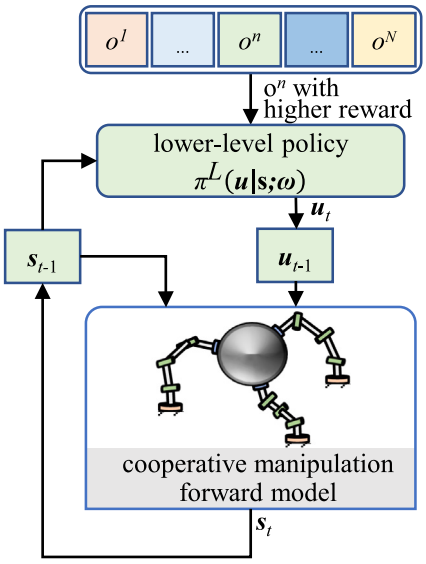


Fig. 4. Schematic of forward prediction by cooperative manipulation probability dynamics model.

The dual function $g\left(\eta_L, \boldsymbol{\theta}_L\right)$ is convex in η_L and $\boldsymbol{\theta}_L$, so the corresponding gradient can be obtained in the form of closed solutions (Peters et al., 2010)

$$\frac{\partial g\left(\eta_L, \boldsymbol{\theta}_L\right)}{\partial \boldsymbol{\theta}_L} = \hat{\boldsymbol{\varphi}}_0 - \frac{\iint_{o,\boldsymbol{\omega}} q\left(\boldsymbol{o}, \boldsymbol{\omega}\right) \exp(V(\boldsymbol{o})) d\boldsymbol{\omega} d\boldsymbol{o} \boldsymbol{\varphi}\left(\boldsymbol{o}\right)}{\iint_{o,\boldsymbol{\omega}} q\left(\boldsymbol{o}, \boldsymbol{\omega}\right) \exp(V(\boldsymbol{o})) d\boldsymbol{\omega} d\boldsymbol{o}}, \tag{39}$$

$$\begin{aligned} \frac{\partial g\left(\eta_L, \boldsymbol{\theta}_L\right)}{\partial \eta_L} &= \varepsilon + \log\left(\iint_{o,\boldsymbol{\omega}} q\left(\boldsymbol{o}, \boldsymbol{\omega}\right) \exp(V(\boldsymbol{o})) d\boldsymbol{\omega} d\boldsymbol{o}\right) \\ &+ \frac{\left(R_{o\boldsymbol{\omega}} - V\left(\boldsymbol{o}\right)\right) \iint_{o,\boldsymbol{\omega}} q\left(\boldsymbol{o}, \boldsymbol{\omega}\right) \exp(V(\boldsymbol{o})) d\boldsymbol{\omega} d\boldsymbol{o}}{\eta_L \iint_{o,\boldsymbol{\omega}} q\left(\boldsymbol{o}, \boldsymbol{\omega}\right) \exp(V(\boldsymbol{o})) d\boldsymbol{\omega} d\boldsymbol{o}}. \end{aligned} \tag{40}$$

According to the gradient results of (39) and (40), the optimal solution can be effectively calculated using any standard optimizer (such as BFGS method). In this implementation, the BFGS algorithm (quasi-Newton method that approximates the Hessian matrix) is used to minimize the dual function:

$$\left[\eta_L^*, \boldsymbol{\theta}_L^*\right] = \arg\min_{\eta_L, \boldsymbol{\theta}_L} g\left(\eta_L, \boldsymbol{\theta}_L\right). \tag{41}$$

η_L^* and $\boldsymbol{\theta}_L^*$ are the optimal dual variables that arise from minimizing $g\left(\eta_L, \boldsymbol{\theta}_L\right)$. Here, η_L^* serves as a Lagrange multiplier enforcing the divergence constraint between $p\left(\boldsymbol{o}, \boldsymbol{\omega}\right)$ and $q\left(\boldsymbol{o}, \boldsymbol{\omega}\right)$, while $\boldsymbol{\theta}_L^*$ adjusts the influence of the feature constraint $\boldsymbol{\varphi}(\boldsymbol{o})$. The optimal dual solution is then used to update the contextual RL policy:

$$\pi^*\left(\boldsymbol{\omega}\,|\boldsymbol{o}\right) = \frac{q\left(\boldsymbol{o}, \boldsymbol{\omega}\right) \exp(V^*\left(\boldsymbol{o}\right))}{\int_{\boldsymbol{\omega}} q\left(\boldsymbol{o}, \boldsymbol{\omega}\right) \exp(V^*\left(\boldsymbol{o}\right)) d\boldsymbol{\omega}}, \tag{42}$$

where $V^*(\boldsymbol{o}) = \left(\frac{R_{o\boldsymbol{\omega}} - (\boldsymbol{\theta}_L^*)^T \boldsymbol{\varphi}(\boldsymbol{o})}{\eta_L^*}\right)$ is computed directly from the optimal dual variables η_L^*, $\boldsymbol{\theta}_L^*$ along with the observation $\boldsymbol{o}$, thereby ensuring that the policy update is firmly grounded in the underlying constraints. This direct computation implies that the adjusted value function seamlessly reflects the influence of both the reward structure and the constraints specifically, the feature constraint modulated by $\boldsymbol{\theta}_L^*$ and the divergence constraint enforced by η_L^*. Consequently, when this value function is integrated into the contextual reinforcement learning policy update, it inherently balances the trade-off between maximizing expected rewards and adhering to the regularization constraints derived from the original optimization problem. Moreover, the BFGS optimizer is employed for its proven efficiency in handling non-linear, convex optimization tasks through iterative Hessian approximation, which in turn ensures rapid convergence. Overall, this approach yields a balanced policy that leverages both the prior distribution $q\left(\boldsymbol{o}, \boldsymbol{\omega}\right)$ and the refined value function to facilitate more robust decision-making in complex environments.

3.2. MCRL

The REPS-based CRL is model-free and requires hundreds, if not thousands, of interactions with the environment to converge to a high-quality policy. For robotic cooperative manipulation problems, even more trials are necessary to master simple manipulation skills. This inefficiency is impractical because conducting actual robot experiments is time-consuming and may lead to robot wear and tear or even damage. Employing a model-based approach significantly enhances the efficiency of REPS-based CRL. Therefore, a model-based contextual reinforcement learning approach (MCRL) is presented in this work. A Gaussian Process model is utilized to characterize the forward model in robotic cooperative manipulation. Leveraging these learned models enables the prediction of simulation or experimental outcomes, thereby facilitating more robust and efficient policy updates. Unlike previous work (Kupcsik et al., 2017), due to the complexity of cooperative tasks, the success or failure of manipulations is a primary focus. Furthermore, rollouts of successful manipulations are reused during the model learning process to enhance the effectiveness and accuracy of the learned model. The implementation of the proposed MCRL is described in Algorithm 1.

The forward probability dynamics model of cooperative manipulation is implemented as a Gaussian process (GP)

$$\boldsymbol{s}_t = \boldsymbol{\Gamma}\left(\boldsymbol{s}_{t-1}, \boldsymbol{u}_{t-1}\right) + \boldsymbol{\epsilon}, \tag{43}$$

where $\boldsymbol{\epsilon} \sim N\left(\mathbf{0}, \boldsymbol{\Sigma}_\varepsilon\right)$ denotes zero-mean Gaussian noise. In order to simplify the learning task, the forward model is decomposed into simpler models, which are easier to learn. For learning the simpler models, the cooperative manipulation model takes tuples $(\boldsymbol{s}_{t-1}, \boldsymbol{u}_{t-1})$ as training inputs and takes $\boldsymbol{s}_t$ as the target output. Typically, tuples corresponding to higher reward contexts are used to more accurately estimate the model and mitigate the impact of manipulation failures in certain contexts. The estimated forward model can generate an artificial trajectory in specific context o^n as instructed in Fig. 4, and the expected episodic reward can be also computed by (27). At each forward prediction step, the actuation action $\boldsymbol{u}_t$ is sampled from the lower-level policy $\pi^L\left(\boldsymbol{u}\,|\boldsymbol{a}\,;\boldsymbol{\omega}\right)$ and the immediate reward $r(\boldsymbol{s}_t, \boldsymbol{u}_t)$ is calculated according the defined cost function.

In each iteration, the initial step involves training the forward probability model using the set of training data. The set is given by $\mathcal{D} = \left\{\boldsymbol{v}^j, \boldsymbol{s}^j\right\}_{j=1}^J$, with $\boldsymbol{v}^j$ and $\boldsymbol{s}^j$ being the training input and target values, where $\boldsymbol{v}^j = [\boldsymbol{s}^j, \boldsymbol{u}^j]$. The dataset is generally sourced from either prior training data or the interaction results of actual cooperative manipulations. Reuse $J(J < H)$ groups' data with highest episodic reward per iteration to form a fresh dataset for model training. For a new input $\boldsymbol{v}^*$, the predictive distribution of the posterior Gaussian process, $P\left(\boldsymbol{s}^*\,|\boldsymbol{v}^*, \mathcal{D}\right)$ is derived

$$p\left(\boldsymbol{s}^*\,|\boldsymbol{v}^*, \mathcal{D}\right) = \mathcal{N}\left(\mu_t\left(\boldsymbol{v}^*\right), \sigma_t^2\left(\boldsymbol{v}^*\right) + \sigma_{noise}^2\right), \tag{44}$$

where mean $\mu_t\left(\boldsymbol{v}^*\right)$ and variance $\sigma_t^2\left(\boldsymbol{v}^*\right)$ are defined as

$$\mu_t\left(\boldsymbol{v}^*\right) = \boldsymbol{k}^T\left[\boldsymbol{K} + \sigma_{noise}^2 \boldsymbol{I}\right]^{-1} \boldsymbol{s}, \tag{45}$$

$$\sigma_t^2\left(\boldsymbol{v}^*\right) = k\left(\boldsymbol{v}^*, \boldsymbol{v}^*\right) - \boldsymbol{k}^T\left[\boldsymbol{K} + \sigma_{noise}^2 I_N\right]^{-1} \boldsymbol{k}, \tag{46}$$

they can be optimized via the expectation–maximization algorithm (Kupcsik et al., 2017; Delgado-Guerrero et al., 2020). $\boldsymbol{I}$ is the identity matrix, $\boldsymbol{K}$ is the $J \times J$ kernel matrix with elements $\boldsymbol{K}^{ij} = k(\boldsymbol{v}^i, \boldsymbol{v}^j), i, j = 1, \ldots, J$, and $\boldsymbol{k}$ is the $J \times 1$ kernel matrix with elements $\boldsymbol{k}^j = k(\boldsymbol{v}^*, \boldsymbol{v}^j)$. The parameter $\sigma_{noise}^2 \boldsymbol{I}$ represents the variance of the system noise $\boldsymbol{\epsilon}$. The

squared exponential function is used to define the covariance function

$$k\left(\boldsymbol{v}^i, \boldsymbol{v}^j\right) = \sigma^2 \exp\left(-\frac{\left(\boldsymbol{v}^i - \boldsymbol{v}^j\right)^T \boldsymbol{L}\left(\boldsymbol{v}^i - \boldsymbol{v}^j\right)}{2}\right), \tag{47}$$

where $\boldsymbol{L} = diag(\boldsymbol{l}^2)$ represents the diagonal matrix with the squared exponential kernel's bandwidth parameters and σ^2 is the kernel variance parameter. $\boldsymbol{l}$, $\sigma^2_{noise} I_N$, and σ are the hyper-parameters of the GP. To obtain accurate predictions, it is essential to appropriately configure the hyperparameters.

Algorithm 1 MCRL Algorithm

Require:
Initial policy $\pi_1(\boldsymbol{\omega}|\boldsymbol{o})$; relative entropy bound ε; number of Iterations K; number of prediction episodes N; number of data collection episodes H; number of episodes with higher reward J.

Ensure: Policy $\pi^*(\boldsymbol{\omega}|\boldsymbol{o})$

1: **for** $k(k = 1, \cdots, K)$ (iterations) **do**
2: Observe H contexts $\left\{\boldsymbol{o}^h\right\}_{h=1}^H$
3: Execute task with $\left\{\boldsymbol{\omega}^h\right\}_{h=1}^H \sim \pi_k\left(\boldsymbol{\omega}|\boldsymbol{o}^h\right)$ and receive reward $\left\{R^h_{o\omega}\right\}_{h=1}^H$
4: Learn forward model using J episodes with higher rewards
5: Draw N contexts $\{\boldsymbol{o}^n\}_{n=1}^N$
6: Execute the current policy with $\pi_k(\boldsymbol{\omega}|\boldsymbol{o}^n)$, get the lower-level policy parameters $\{\boldsymbol{\omega}^n\}_{n=1}^N \sim \pi_k(\boldsymbol{\omega}|\boldsymbol{o}^n)$
7: Predict the state-action set $\{\boldsymbol{\tau}^n\}_{n=1}^N$ with the learned model, calculate the corresponding reward $\left\{R^n_{o\omega}\right\}_{n=1}^N$
8: Calculate the dual function $g\left(\eta_L, \boldsymbol{\theta}_L\right)$ according to (38) with the artificial dataset
9: Compute the gradient of the dual function with respect to $\boldsymbol{\theta}_L$ and η_L according to (39) and (40)
10: Calculate the optimal solution of the dual function $\boldsymbol{\theta}^*_L, \eta^*_L$ with (41)
11: Calculate the corresponding the probability distribution $\left\{p^j\right\}_{i=1}^N$ according to (35)
12: Update the contextual policy $\pi_{k+1}(\boldsymbol{\omega}|\boldsymbol{o})$ with (42)
13: Set $\pi^*(\boldsymbol{\omega}|\boldsymbol{o}) = \pi_{k+1}(\boldsymbol{\omega}|\boldsymbol{o})$
14: **end for**

4. Simulations and experiments

To comprehensively evaluate the algorithm proposed in this paper, cooperative manipulation experiments were conducted involving spherical objects of varying masses to achieve diverse motions. These experiments were carried out using three robotic arms in simulations and two robotic arms in physical experiments. Subsequently, the proposed MCRL algorithm is compared with several other CRL methods, including CREPS and CREPS-CMA, to demonstrate its superiority. Each robotic arm's end-effector is equipped with a force sensor for obtaining wrench information, while employing disk-shaped actuators to directly interact with the manipulated object. Assuming that the shape and position of the manipulated object to be manipulated can be directly determined via remote measurement devices, such as visual cameras, the grasp matrix can be readily obtained through contact point planning. For enhanced validation of the proposed algorithm, it is beneficial to focus solely on translational motion while disregarding the effects of rotation. In the framework of cooperative manipulation, the state space, action space, reward function, and context space of MCRL in both simulations and experiments are consistent, and they can be defined as follows

(1) State Space, the state $\boldsymbol{s}$ of the manipulated object, comprised of pose wrench components, can be expressed as

$$\boldsymbol{s} = \left[\boldsymbol{x}, \boldsymbol{u}^C\right] \in \mathbb{R}^{6+6i}, \tag{48}$$

where $x \in \mathbb{R}^6$ denotes the pose of the manipulated object, $\boldsymbol{u}^C \in \mathbb{R}^6$ is the constrained wrenches exerted on the manipulated object. Minimizing the magnitude of constrained wrenches is advantageous for energy conservation while ensuring grasp constraints. When solely considering the movement of the manipulated object and the constraint wrenches along the normal direction of the contact surface, the observable state $\boldsymbol{s}$ can be efficiently represented in a reduced dimension as

$$\boldsymbol{s} = \left[x, y, z, f^C_{1,z}, \ldots, f^C_{i,z}\right] \in \mathbb{R}^{3+i}, \tag{49}$$

where x, y, and z correspond to the position of the manipulated object in the Cartesian space. For computational simplification, it is common practice to set the initial position as the coordinate origin. This consistency is maintained throughout subsequent discussions and in the design of action and context spaces related to the dimensionally reduced state $\boldsymbol{s}$.

(2) Action Space, the actuation action in this study refer to the contact wrenches applied to the manipulated object, it is defined as

$$\boldsymbol{u} = [\boldsymbol{f}_1, \boldsymbol{f}_2, \ldots, \boldsymbol{f}_i]^T = \boldsymbol{u}^U + \boldsymbol{u}^C \in \mathbb{R}^{3i}. \tag{50}$$

By introducing parameterized decoupled lower-level control policy, the actuation $\boldsymbol{u}$ can be decomposed into the unconstrained actuation $\boldsymbol{u}^U$ and the constrained actuation $\boldsymbol{u}^C$, computed individually according to (14) and (15), respectively. The parameters $\boldsymbol{\omega} \in \mathbb{R}^{7+i}$ of the decoupled controller in (21) need to be learned and updated based on contextual features $\boldsymbol{o}$.

(3) Reward Function, given parameter vector $\boldsymbol{\omega}$ executed in context $\boldsymbol{o}$, the episodic reward $R(\boldsymbol{\tau}, \boldsymbol{o})$ is defined as

$$\begin{aligned} r(\boldsymbol{s}, \boldsymbol{u}) = &- c_1(\boldsymbol{x} - \boldsymbol{x}_d)^T(\boldsymbol{x} - \boldsymbol{x}_d) \\ &- c_2(\boldsymbol{u}^{C,z})^T\boldsymbol{u}^{C,z} - c_3\psi(\boldsymbol{s}), \end{aligned} \tag{51}$$

$$\boldsymbol{u}^{C,z} = \left[f^C_{1,z}, \ldots, f^C_{i,z}\right]^T, \tag{52}$$

$$\psi(\boldsymbol{s}_t) = \begin{cases} 1, & \text{grasp constraints satisfied} \\ 0, & \text{grasp constraints unsatisfied} \end{cases}, \tag{53}$$

where the first term aims to minimize the deviation between the actual position $\boldsymbol{x}$ and the desired position $\boldsymbol{x}_d$ of the manipulated object, where $\boldsymbol{x}_d$ can be computed based on the predetermined desired acceleration $\ddot{\boldsymbol{x}}_d$; the objective of the second term is to minimize the constraint wrench $\boldsymbol{u}^{C,z}$ exerted on the manipulated object, namely internal forces, in order to conserve energy; the third term serves as the primary performance metric, ensuring the success of the cooperative manipulation task by enforcing grasp constraints. A successful task execution requires maintaining a stable grasp throughout the operation, and this is explicitly reflected in the penalty function $\psi(\boldsymbol{s}_t)$. Hence, (27) is utilized to compute the expected episode reward. In cooperative manipulation tasks, achieving success takes precedence over specific performance metrics like trajectory tracking and minimal internal forces. Therefore, the coefficient c_3 is generally set to be significantly larger than c_1 and c_2, reinforcing the priority of grasp stability over other optimization objectives.

(4) Context Space, the desired acceleration $\ddot{\boldsymbol{x}}_d$ and the mass m of manipulated are selected to form the context $\boldsymbol{o} = [\ddot{\boldsymbol{x}}_d, m] \in \mathbb{R}^4$. The selection of acceleration over velocity or position is based on its fundamental role in the dynamic modeling of robotic manipulation. According to Newton's second law, wrench is directly proportional to acceleration, making it a crucial parameter for precise wrench and motion control. By incorporating acceleration into the context, the model can more effectively capture system dynamics and adapt to real-time changes. Similarly, the inclusion of mass mm as a context variable is motivated by its role in the cooperative manipulation framework. In this framework, control is transformed into constrained and unconstrained subspaces via the grasp matrix, inherently reducing the influence of the object's shape on the overall manipulation process. As a result, mass serves as a more representative characteristic of the manipulated object, effectively capturing its inertial properties and facilitating accurate wrench and motion planning. The rationale

Table 1
Simulation and experiment parameter table.

	Parameter name	Symbol	Value	
			Simulation	Experiments
Manipulated object parameters	Mass	m	$1 \leq m \leq 2$ kg	1 kg/2 kg
	Sphere diameter	d	0.2 m	0.2 m
	Coefficient of friction	μ_f	0.9	$0.4 \leq \mu_f \leq 0.8$
Initial Values for lower-level controller parameters	Proportional gain for unconstrainted wrenches	$\boldsymbol{k}_P$	[0.1, 0.1, 0.1]	[0.1, 0.1, 0.1]
	Derivative gain for unconstrainted wrenches	$\boldsymbol{k}_D$	[0.05, 0.05, 0.05]	[0.05, 0.05, 0.05]
	Baseline term for constrainted wrenches	$\boldsymbol{f}^C_{basis}$	2 N	2 N
	Proportional term for constrainted wrenches	k_{int}	5	5
CRL Parameters	Number of prediction episodes	N	1000	20
	Total Number of Iterations	K	100	20
	Upper Bound of Relative Entropy	ϵ	0.3	0.3
Reward function parameters	Position deviation minimization coefficient	c_1	1	1
	Force minimization coefficient	c_2	0.5	0.5
	Penalty term coefficient	c_3	1000	1000

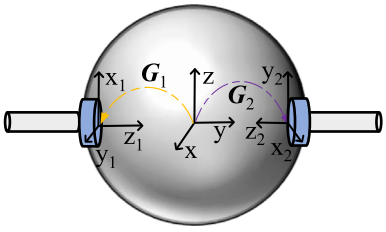


Fig. 5. Grasping configuration of dual-arm manipulation.

behind selecting these context variables is to enable the robotic cooperative system to autonomously acquire skills for manipulating objects with diverse physical properties and movement requirements. Consequently, tasks associated with different contexts will lead to distinct reward functions, allowing the system to adapt dynamically to varying manipulation scenarios.

4.1. Simulations

The proposed method was validated through simulations involving tasks with dual-arm and multi-arm robotic systems. In these tasks, spheres of varying masses were manipulated to achieve different acceleration profiles. The simulation parameters, applicable to the contextual reinforcement learning (CRL) methods including MCRL, CREPS, and CREPS-CMA, are configured as depicted in Table 1. Since these CRL methods are based on Relative Entropy Policy Search (REPS), their specific application processes are fundamentally similar to the MCRL approach highlighted in this paper. The MCRL algorithm notably incorporates a forward Gaussian model, utilizing the top-performing H data sets from each iteration to train a more accurate forward probability model, thereby enhancing training performance. The designs of CREPS and CREPS-CMA rely on the parameters listed in Table 1. Specifically, CREPS-CMA updates the covariance matrix based on data from each iteration, a process that does not require additional parameters beyond those provided in Table 1 (Abdolmaleki et al., 2016b). Within these CRL algorithm frameworks, simulations were conducted over a duration of 4 s with a time step of 0.01 s to train and obtain lower-level policies. The context variable o has values ranging from mass $1\text{kg} < m < 5\text{kg}$, with absolute acceleration values in each direction not exceeding 1 m/s^2. The maximum wrench input f_i for the robotic arm in each direction does not exceed 5N.

In cooperative dual-arm manipulation tasks, the configuration of the end-effector grasping is depicted in Fig. 5. The grasp matrix for dual-arm manipulation is formulated as $\boldsymbol{G} = [\boldsymbol{G}_1, \boldsymbol{G}_2]$. Parameterized decoupled lower-level policy design is carried out based on this grasping matrix. Using CRL, policy parameters are generated for different contexts to perform cooperative manipulation tasks. The reward and success rate of these dual-arm manipulation tasks are shown in Fig. 6(a) and Fig. 6(b). In these figures, the proposed MCRL algorithm is represented by a solid blue line, while CREPS and CREPS-CMA are depicted using an orange dashed line and a green dotted line, respectively. It is evident that the proposed MCRL algorithm can perform much better than CREPS and CREPS-CMA. It can achieve significant average rewards after just 10 iterations in Fig. 6(b) and has a higher success rate in cooperative manipulations. While CREPS-CMA may achieve higher rewards after 100 iterations, its success rate in task execution remains relatively low. In contrast, CREPS exhibits greater reward fluctuations; however, its success rate consistently exceeds 90%, a significant improvement compared to CREPS-CMA. This highlights that due to the complexity of robot cooperative manipulations, relying solely on reward variations during contextual skill learning and generalization may not fully capture learning effectiveness. MCRL utilizes data with higher rewards for model learning, thereby improving learned model accuracy and ensuring both the success rate and per-episode performance.

In multi-arm cooperative manipulation tasks, Fig. 7 provides a top-down view of the end-effector grasping configuration, with a line from the robotic arm end-effectors to the sphere's center forming an angle of 120°. The grasp matrix for multi-arm manipulation is computed with $\boldsymbol{G} = [\boldsymbol{G}_1, \boldsymbol{G}_2, \boldsymbol{G}_3]$. Fig. 8 displays the rewards obtained and the corresponding changes in success rates for the MCRL, CREPS, and CREPS-CMA methods. After only 10 iterations, MCRL demonstrates notably superior performance compared to the other two CRL methods in multi-arm cooperative manipulations. A portion of the data is specifically reserved to train MCRL's forward Gaussian model before the iterations, significantly enhancing early-stage learning performance, especially the success rate as shown in Fig. 8(b). CREPS experiences some fluctuations in both average episode rewards and success rates, whereas CREPS-CMA demonstrates a convergence to lower levels in both metrics. This emphasizes that integrating model-based learning algorithms can effectively strengthens both the generalization capability and stability of CRL algorithms in complex task manipulations.

In both dual-arm and multi-arm cooperative manipulation skill learning processes, MCRL consistently outperforms CREPS and CREPS-CMA. The benefits of the proposed algorithm are particularly evident

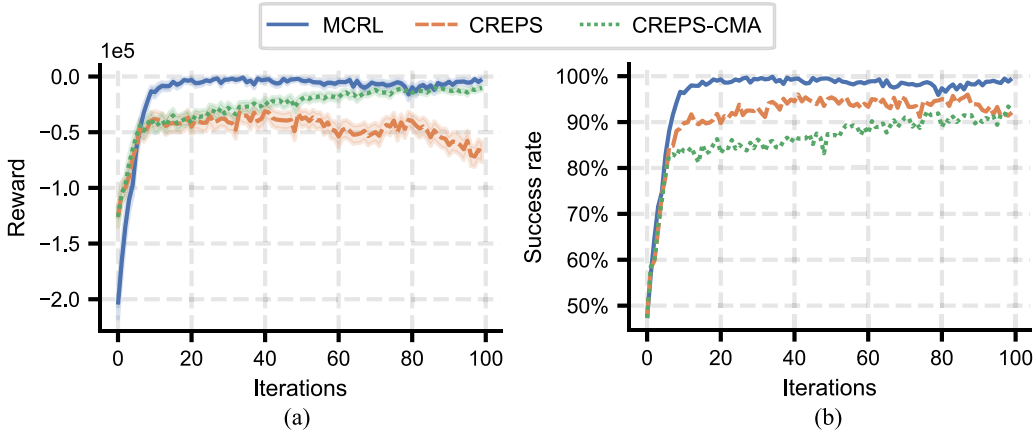


Fig. 6. Performance on dual-arm cooperative manipulation tasks for MCRL, CREPS, and CREPS-CMA in simulations. (a) Episode reward over iterations. (b) Success rate of cooperative manipulation over iterations.

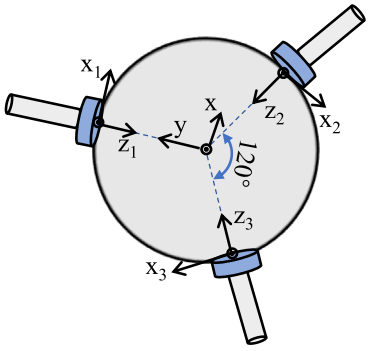


Fig. 7. Grasping configuration of multi-arm manipulation.

in multi-arm cooperative manipulations, as demonstrated by the average episode rewards and success rates shown in Figs. 6 and 8. Furthermore, MCRL shows promise in mitigating increased uncertainty arising from contact dynamics and external disturbances as manipulation modes grow more complex, especially in generalizing robot cooperative manipulation skills. Beyond these performance advantages, the improved efficiency and resource utilization of the MCRL approach are also clearly reflected in the simulation results. Specifically, the forward Gaussian model effectively pre-selects high-quality data, which accelerates policy convergence and reduces the number of required iterations. Moreover, the decoupled controller design minimizes redundant computations, leading to lower energy consumption and more efficient use of computational resources. Consequently, the proposed MCRL approach facilitates safer and more stable skill generalization while efficiently achieving high-quality policies.

4.2. Experiments

A dual-arm cooperative manipulation platform is set up as shown in Fig. 9. Each UR5 robotic arm is equipped with a six-axis force sensor attached to its end effector, measuring the contact wrenches between the end effector and the objects being manipulated at a sampling rate of 100 Hz. The specific force sensor models used are the ATI NC-Axia80-M20 and the OnRobot HEX-E QC. Control commands are generated in real time based on the force sensor data and transmitted to both UR5 robots via TCP/IP. The sphere diameter and friction coefficient of the manipulated object sphere are depicted in Table 1. The friction coefficient μ_f is challenging to measure accurately. Using the sliding measurement method, its value has been determined to fall within the range of 0.4 to 0.8 due to the change in contact area (Chey et al., 2016). For sensor calibration, the built-in calibration commands of the ATI force sensor were utilized to zero the wrench reading before contact, establish a contact force threshold, and minimize the impact of measurement noise, thereby reducing the likelihood of false contact detection (Wang et al., 2021). The low-level policy parameters acquired through CRL according to the fixed contexts. Additionally, details regarding sensor calibration, control frequency, and communication latency have been explicitly included to enhance the reproducibility and transparency of the experimental setup. Constrained force tracking control is implemented using Proportional-Derivative (PD) control based on measurement information from the force sensor, while position tracking control is achieved using the built-in position controller of the UR5 robot. This framework validates the proposed parameter decoupling in lower-level control and facilitates the verification of the CRL policy.

In the experiments, the policies learned by simulations are implemented on the physical robot, and the CRL policies are updated and evaluated according to the actual experiment data set. Specifically, the robot cooperative manipulation skills learning with MCRL in experiment is shown in Fig. 10. MCRL learns the model through the interactions between the robotic arms and the manipulated objects. This data-driven modeling approach, grounded in actual interactions, mitigates inaccuracies resulting from imperfect model tuning and a scarcity of precise expert knowledge. Moreover, it continuously updates model parameters by integrating the latest measurement data to refine the learned model. To ensure data quality, only a few sets with high reward evaluations are employed for model learning. Utilizing the learned model, artificial samples in computer simulations undergo evaluation to ensure the safety of cooperative manipulation. Regarding the other two methods, CREPS and CREPS-CMA, they also employ a combination of simulations and physical experiments for policy iteration. The contexts of cooperative manipulation failure are only realized in the simulations to ensure the safety of the experiments.

The experiment is conducted on an object weighing 1kg, with 1000 contexts in simulations and 20 contexts in experiments per iteration. Other experiment parameters are set as Table 1. Considering the workspace of the UR5 robot, the accelerations are constrained within the range of $[-0.01, 0.01]$ m/s^2. Based on the current high-level policy, a low-level policy is generated to perform the cooperative manipulation task experiments. The learning curves are shown in Fig. 11 including average reward curves and success rate curves over iterations. During the policy update phase in the simulation, it is obvious that the average rewards and manipulation success rates of the MCRL, CREPS, and CREPS-CMA methods converge and stabilize relatively quickly. The initially low manipulation success rate is attributed to the uncertainty caused by friction factors. For the 20 randomly generated contexts, the force profiles generated by the policy obtained from the simulation

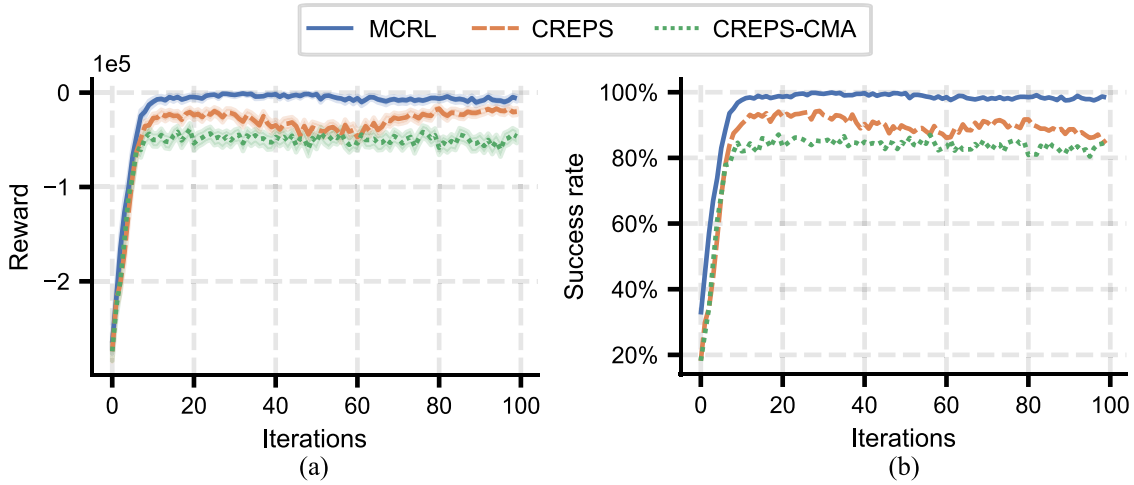


Fig. 8. Performance on multi-arm cooperative manipulation tasks for MCRL, CREPS, and CREPS-CMA in simulations. (a) Episode reward over iterations. (b) Success rate of cooperative manipulation over iterations.

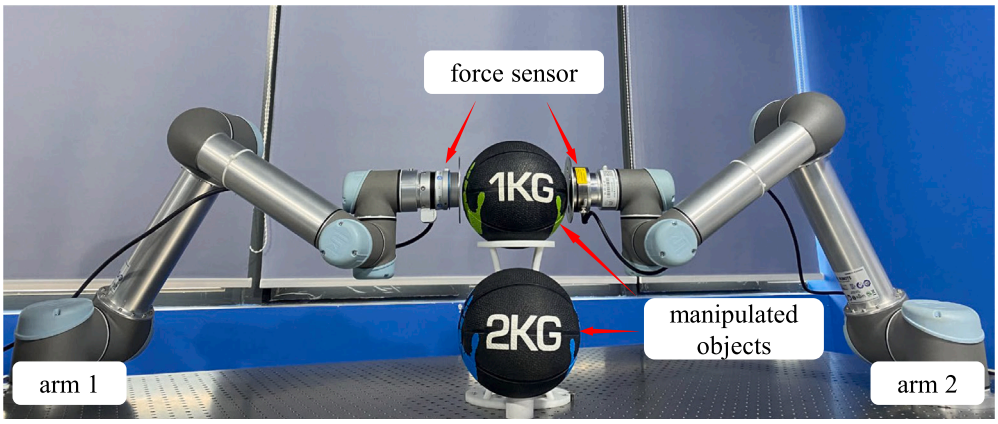


Fig. 9. Dual-arm cooperative manipulation platform.

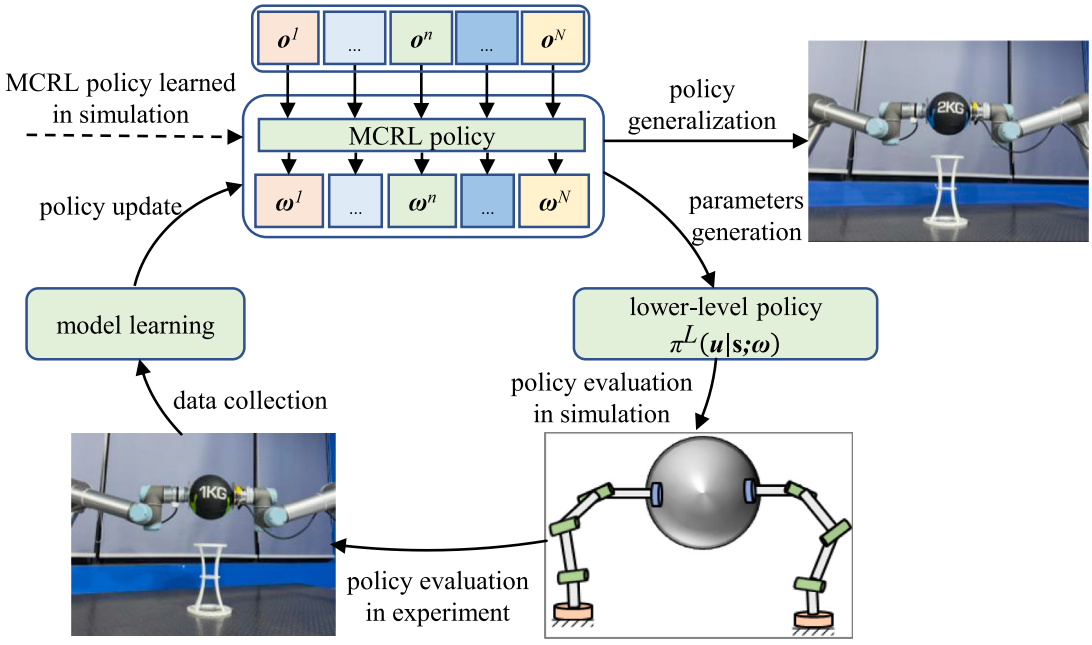


Fig. 10. Schematic of robot cooperative manipulation skill learning in experiment.

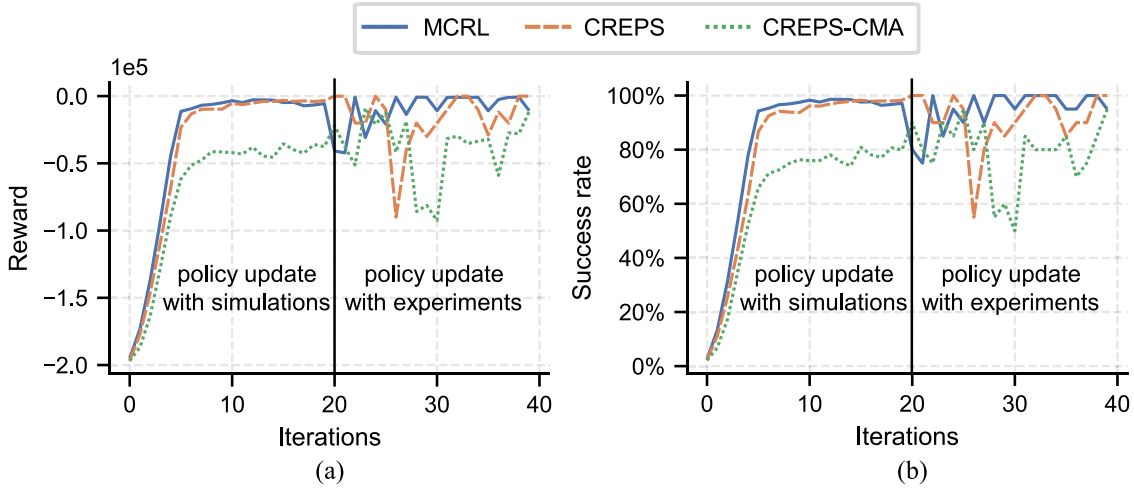


Fig. 11. Performance on dual-arm cooperative manipulation tasks for MCRL, CREPS, and CREPS-CMA in experiments. (a) Average reward curve for the experiment over iterations. (b) Success rate of cooperative manipulation over iterations.

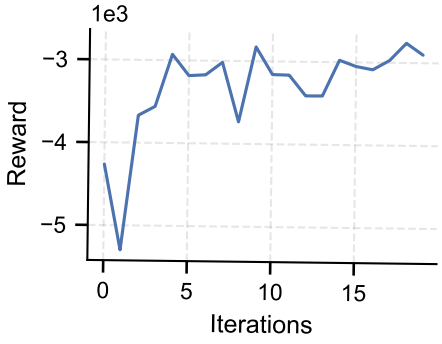


Fig. 12. Average reward curve for $m = 2$ kg and $a = [0.005, 0, 0.01]$ m/s^2 in experiments.

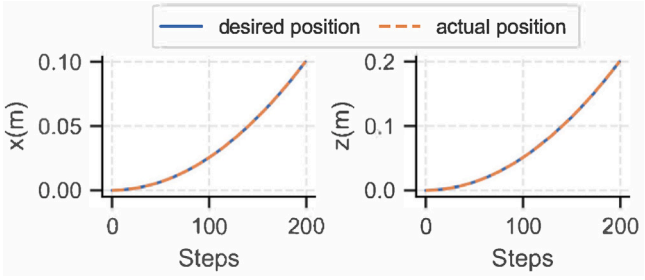


Fig. 13. Position tracking performance. (a) Position tracking along x direction. (b) Position tracking along z direction.

are executed on the UR5 robots using the corresponding lower-level policies. The resulting average reward and manipulation success rate curves during the policy update phase in the experiments are shown in Fig. 11(a) and (b). During the experimental phase, the CRL method exhibits oscillations in average rewards and manipulation success rates due to the uncertainty of environment modeling and external disturbances. However, the proposed MCRL method continuously corrects the environmental model using actual measurement data, thereby achieving and maintaining higher rewards and manipulation success rates. The other two methods clearly lack this ability to resist environmental disturbances.

To further validate the effectiveness and generalization capability of the proposed method, the learned policy as shown in Fig. 11 is extended to manipulate a 2 kg object to achieve motion with an acceleration $a = [0.005, 0, 0.01]$ m/s^2 over the iterations. In this scenario, the y-axis direction of the manipulated object primarily achieves control of constrained forces, while the x and z-axis directions mainly achieve motion control, which can be accomplished through policy parameters for position-based control. Throughout the corresponding 20 experiment iterations, the obtained average rewards are depicted in Fig. 12. There were no instances of manipulation failures throughout the entire experimental manipulation process, demonstrating the robust generalization capability of the proposed method. The tracking performance of the position over multiple iterations is illustrated in Fig. 13, consistently maintaining satisfactory results. Fig. 14(a) displays the learning curve of the desired constraint forces, showing continuous optimization throughout the iterative learning process. The corresponding force tracking performance, as shown in Fig. 14(b), validates that the proposed MCRL algorithm can plan executable and effective lower-level policies while ensuring both constrained force and position tracking performance of the manipulated object.

Furthermore, the experimental outcomes substantiate the efficiency and resource utilization benefits of the MCRL method. Specifically, the approach reduces computational overhead by leveraging decoupled control structures and by continuously updating the model with high-quality data. As a result, energy consumption is lowered, and operation stability is enhanced. These improvements not only contribute to the robust performance observed in simulation but also extend to real-world experiments. Finally, the application of reinforcement learning strategies to cooperative manipulation scenarios is particularly noteworthy. The experimental results demonstrate that the proposed algorithm achieves a high success rate in cooperative manipulation tasks, thereby eliminating the need for dangerous and time-consuming physical training. Simultaneously, it consistently yields high-quality cooperative manipulation strategies, underscoring its practical value.

While these advantages underscore the robustness of MCRL, real-world implementation still presents certain challenges. The success rate in physical trials remains lower than in simulations due to real-world uncertainties, as illustrated in the comparison between Fig. 6(b) and Fig. 11(b). Key contributing factors include sensor noise, which affects force measurement accuracy and control precision; model mismatch, where discrepancies in contact dynamics and friction properties lead to deviations from simulated performance; external disturbances, such as mechanical vibrations and environmental fluctuations that are not fully captured in simulations; and hardware limitations, including actuation delays and real-time computation constraints. Collectively, these factors introduce uncertainties that impact real-world performance. However, the proposed MCRL method continuously refines its model using real-world data, improving adaptability and robustness in practical applications.

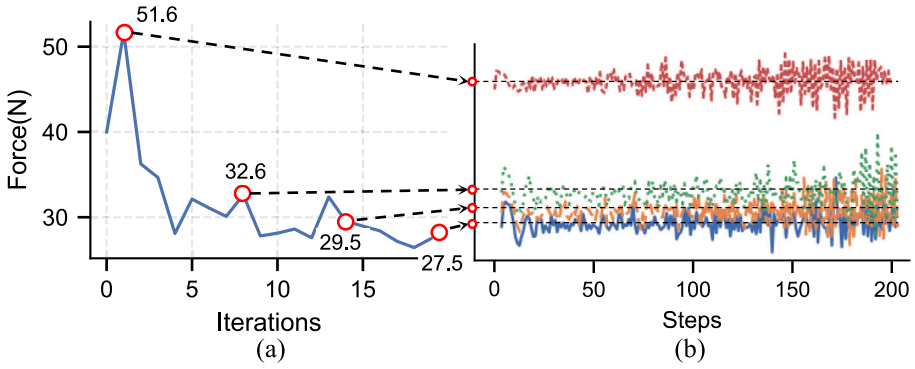


Fig. 14. (a) Desired constrainted force learning curves for $m = 2$ kg and $a = [0.005, 0, 0.01]$ m/s^2 in experiments. (b)Force tracking performance for four specific desired constrainted forces.

5. Conclusion

In this study, the challenge of adapting lower-level controllers to various contexts in cooperative manipulation tasks is addressed. Traditional methods often necessitate complete re-learning when faced with different manipulated objects or reference trajectories due to their fixed-context approach. To overcome this limitation, a model-based contextual reinforcement learning (MCRL) approach was introduced for robotic cooperative manipulation. The MCRL framework leverages a parameterized lower-level control policy based on projected inverse dynamics, allowing for decoupled control of cooperatively manipulated objects across both constrained and unconstrained subspaces. Additionally, a probabilistic forward model was incorporated to compute expected returns and mitigate risk-sensitive bias, optimizing the cooperative manipulation model. The effectiveness of the MCRL approach was demonstrated through comprehensive evaluations on both a multi-robot simulation system and a real dual-arm platform. The results affirm that MCRL not only accomplishes cooperative manipulation tasks but also achieves high-quality policies efficiently. Furthermore, the experimental and simulation results demonstrate that MCRL can improve efficiency and resource utilization. Specifically, the integration of a forward Gaussian model enables the pre-selection of high-quality data, accelerating policy convergence; the decoupled design of the lower-level controller minimizes redundant computations, reducing computational overhead; and continuous model updates enhance robustness to environmental uncertainties, leading to reduced energy consumption and improved operational stability. These findings highlight the potential of MCRL in enhancing the adaptability, stability, and efficacy of robotic cooperative manipulation in diverse contexts.

In summary, the proposed MCRL method offers a promising avenue for generalizing lower-level controllers across different cooperative manipulation scenarios, paving the way for more robust and versatile robotic cooperative manipulations in real-world applications. Nevertheless, MCRL also exhibits certain limitations, including increased computational complexity in high-dimensional settings and sensitivity to hyperparameter choices, which may affect performance in highly dynamic or uncertain environments. Future enhancements may focus on developing scalable optimization methods, adaptive hyperparameter tuning strategies, and advanced sensor fusion techniques to further improve the robustness, efficiency, and generalization capabilities of the approach.

CRediT authorship contribution statement

Wenrui Wang: Writing – review & editing, Writing – original draft, Visualization, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Qirong Tang:** Writing – review & editing, Methodology, Project administration, Funding acquisition, Supervision, Resources, Conceptualization. **Hao Yang:** Data curation. **Chao Yang:** Validation, Data curation. **Baoping Ma:** Validation, Data curation. **Shuqing Wang:** Formal analysis, Data curation. **Rongfu Lin:** Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (No. 52305032, 62373285), the “National High Level Overseas Talent Plan”, the “National Major Talent Plan” (No. 2022-XXXX-XXX-079), the Shanghai Pujiang Programme (No. 23PJD103), a fundamental research project (No. XXXX2022YYYC133), and the National Laboratory of Space Intelligent Control (No. HTKJ2023KL502016). All support is greatly appreciated.

Data availability

The authors do not have permission to share data.

References

Abdelali, H.B., Sahnoun, M., Sakka, S., Damak, M., Siarry, P., Kallel, F., 2024. Deep learning-based methods for MS lesion segmentation: A review. In: 2024 IEEE 7th International Conference on Advanced Technologies, Signal and Image Processing, Vol. 1. ATSIP, IEEE, pp. 676–682.

Abdolmaleki, A., Lau, N., Reis, L.P., Peters, J., Neumann, G., 2016a. Contextual policy search for linear and nonlinear generalization of a humanoid walking controller. J. Intell. Robot. Syst. 83, 393–408.

Abdolmaleki, A., Lioutikov, R., Peters, J.R., Lau, N., Pualo Reis, L., Neumann, G., 2015. Model-based relative entropy stochastic search. Adv. Neural Inf. Process. Syst. 28.

Abdolmaleki, A., Simões, D., Lau, N., Reis, L.P., Neumann, G., 2016b. Contextual relative entropy policy search with covariance matrix adaptation. In: 2016 International Conference on Autonomous Robot Systems and Competitions. ICARSC, IEEE, pp. 94–99.

Antonova, R., Yang, J., Sundaresan, P., Fox, D., Ramos, F., Bohg, J., 2022. A bayesian treatment of real-to-sim for deformable object manipulation. IEEE Robot. Autom. Lett. 7 (3), 5819–5826.

Buchli, J., Stulp, F., Theodorou, E., Schaal, S., 2011. Learning variable impedance control. Int. J. Robot. Res. 30 (7), 820–833.

Buşoniu, L., De Bruin, T., Tolić, D., Kober, J., Palunko, I., 2018. Reinforcement learning for control: Performance, stability, and deep approximators. Annu. Rev. Control. 46, 8–28.

Chatzilygeroudis, K., Vassiliades, V., Stulp, F., Calinon, S., Mouret, J.-B., 2019. A survey on policy search algorithms for learning robot controllers in a handful of trials. IEEE Trans. Robot. 36 (2), 328–347.

Chey, S.K., Tian, P., Tian, Y., 2016. Estimation of real contact area during sliding friction from interface temperature. Aip Adv. 6 (6).

Culbertson, P., Slotine, J.-J., Schwager, M., 2021. Decentralized adaptive control for collaborative manipulation of rigid bodies. IEEE Trans. Robot. 37 (6), 1906–1920.
Daniel, C., Neumann, G., Kroemer, O., Peters, J., 2016. Hierarchical relative entropy policy search. J. Mach. Learn. Res. 17 (93), 1–50.
Dehio, N., Smith, J., Wigand, D.L., Xin, G., Lin, H.-C., Steil, J.J., Mistry, M., 2018. Modeling and control of multi-arm and multi-leg robots: Compensating for object dynamics during grasping. In: 2018 IEEE International Conference on Robotics and Automation. ICRA, IEEE, pp. 294–301.
Deisenroth, M.P., Fox, D., Rasmussen, C.E., 2013. Gaussian processes for data-efficient learning in robotics and control. IEEE Trans. Pattern Anal. Mach. Intell. 37 (2), 408–423.
Delgado-Guerrero, J.A., Colomé, A., Torras, C., 2020. Contextual policy search for micro-data robot motion learning through covariate gaussian process latent variable models. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, IEEE, pp. 5511–5517.
Dohmann, P.B.G., Lederer, A., Dißemond, M., Hirche, S., 2021. Distributed Bayesian online learning for cooperative manipulation. In: 2021 60th IEEE Conference on Decision and Control. CDC, IEEE, pp. 2888–2895.
Erhart, S., Hirche, S., 2015. Internal force analysis and load distribution for cooperative multi-robot manipulation. IEEE Trans. Robot. 31 (5), 1238–1243.
Fanger, Y., Umlauft, J., Hirche, S., 2016. Gaussian processes for dynamic movement primitives with application in knowledge-based cooperation. In: 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, IEEE, pp. 3913–3919.
Gueaieb, W., Karray, F., Al-Sharhan, S., 2003. A robust adaptive fuzzy position/force control scheme for cooperative manipulators. IEEE Trans. Control Syst. Technol. 11 (4), 516–528.
Hou, Z., Fei, J., Deng, Y., Xu, J., 2020. Data-efficient hierarchical reinforcement learning for robotic assembly control applications. IEEE Trans. Ind. Electron. 68 (11), 11565–11575.
Hou, Z., Yang, W., Chen, R., Feng, P., Xu, J., 2022. A hierarchical compliance-based contextual policy search for robotic manipulation tasks with multiple objectives. IEEE Trans. Ind. Informatics 19 (4), 5444–5455.
Jang, D., Yoo, J., Son, C.Y., Kim, D., Kim, H.J., 2020. Multi-robot active sensing and environmental model learning with distributed Gaussian process. IEEE Robot. Autom. Lett. 5 (4), 5905–5912.
Jean, J.H., Fu, L.C., 1993. An adaptive control scheme for coordinated multimanipulator systems. IEEE Trans. Robot. Autom. 9 (2), 226–231.
Jiang, R., Ye, D., Xiao, Y., Sun, Z., Zhang, Z., 2023. Orbital interception pursuit strategy for random evasion using deep reinforcement learning. Space: Sci. Technol. 3, 0086.
Jiao, C., Yu, L., Su, X., Wen, Y., Dai, X., 2022. Adaptive hybrid impedance control for dual-arm cooperative manipulation with object uncertainties. Automatica 140, 110232.
Jin, L., Men, Y., Li, F., Wang, C., Tian, X., Li, Y., Song, R., 2024. Ensemble transfer strategy based on domain difference for robot multiple peg-in-hole assembly. IEEE Trans. Ind. Electron..
Junge, K., Hughes, J., Thuruthel, T.G., Iida, F., 2020. Improving robotic cooking using batch Bayesian optimization. IEEE Robot. Autom. Lett. 5 (2), 760–765.
Kawasaki, H., Ueki, S., Ito, S., 2006. Decentralized adaptive coordinated control of multiple robot arms without using a force sensor. Automatica 42 (3), 481–488.
Klink, P., Abdulsamad, H., Belousov, B., Peters, J., 2020. Self-paced contextual reinforcement learning. In: Conference on Robot Learning. PMLR, pp. 513–529.
Kong, L., He, W., Yang, C., Li, Z., Sun, C., 2019. Adaptive fuzzy control for coordinated multiple robots with constraint using impedance learning. IEEE Trans. Cybern. 49 (8), 3052–3063.
Kroemer, O., Niekum, S., Konidaris, G., 2021. A review of robot learning for manipulation: Challenges, representations, and algorithms. J. Mach. Learn. Res. 22 (30), 1–82.
Kulkarni, A.J., Kale, I.R., Shastri, A., Khandekar, A., 2024. Snail homing and mating search algorithm: a novel bio-inspired metaheuristic algorithm. Soft Comput. 1–40.
Kumar, V., Hermans, T., Fox, D., Birchfield, S., Tremblay, J., 2019. Contextual reinforcement learning of visuo-tactile multi-fingered grasping policies. arXiv preprint arXiv:1911.09233.
Kupcsik, A., Deisenroth, M.P., Peters, J., Loh, A.P., Vadakkepat, P., Neumann, G., 2017. Model-based contextual policy search for data-efficient generalization of robot skills. Artificial Intelligence 247, 415–439.
Kupcsik, A., Deisenroth, M., Peters, J., Neumann, G., 2013. Data-efficient generalization of robot skills with contextual policy search. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 27, (No. 1), pp. 1401–1407.
Lin, H.C., Smith, J., Babarahmati, K.K., Dehio, N., Mistry, M., 2018. A projected inverse dynamics approach for multi-arm cartesian impedance control. In: 2018 IEEE International Conference on Robotics and Automation. ICRA, IEEE, pp. 5421–5428.
Mistry, M., Righetti, L., 2012. Operational space control of constrained and underactuated systems.
Murray, R.M., Li, Z., Sastry, S.S., 2017. A Mathematical Introduction to Robotic Manipulation. CRC Press.
Panwar, V., Kumar, N., Sukavanam, N., Borm, J.H., 2012. Adaptive neural controller for cooperative multiple robot manipulator system manipulating a single rigid object. Appl. Soft Comput. 12 (1), 216–227.
Peters, J., Mulling, K., Altun, Y., 2010. Relative entropy policy search. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 24, (No. 1), pp. 1607–1612.
Pong, V., Gu, S., Dalal, M., Levine, S., 2018. Temporal difference models: Model-free deep rl for model-based control. arXiv preprint arXiv:1802.09081.
Righetti, L., Buchli, J., Mistry, M., Kalakrishnan, M., Schaal, S., 2013. Optimal distribution of contact forces with inverse-dynamics control. Int. J. Robot. Res. 32 (3), 280–298.
Song, Y., Scaramuzza, D., 2022. Policy search for model predictive control with application to agile drone flight. IEEE Trans. Robot. 38 (4), 2114–2130.
Spong, M.W., Hutchinson, S., Vidyasagar, M., 2020. Robot Modeling and Control. John Wiley & Sons.
Tangkaratt, V., Van Hoof, H., Parisi, S., Neumann, G., Peters, J., Sugiyama, M., 2017. Policy search with high-dimensional context variables. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 31, (No. 1).
Trinkle, J.C., Pang, J.-S., Sudarsky, S., Lo, G., 1997. On dynamic multi-rigid-body contact problems with Coulomb friction. Z. Für Angew. Math. Und Mech. (ZAMM-J. Appl. Math. Mechanics) 77 (4), 267–279.
Wang, W., Li, Q., Lu, C., Gu, J., Li, A., Li, Y., Huo, Q., Chu, H., Zhu, M., 2021. Impedance estimation for robot contact with uncalibrated environments. Mech. Syst. Signal Process. 159, 107819.
Willemsen, D., Coppola, M., de Croon, G.C., 2021. MAMBPO: Sample-efficient multi-robot reinforcement learning using learned world models. In: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, IEEE, pp. 5635–5640.
Xu, P., Cui, Y., Shen, Y., Zhu, W., Zhang, Y., Wang, B., Tang, Q., 2023. Reinforcement learning compensated coordination control of multiple mobile manipulators for tight cooperation. Eng. Appl. Artif. Intell. 123, 106281.
Yu, C., Cao, J., Rosendo, A., 2022. Learning to climb: Constrained contextual bayesian optimisation on a multi-modal legged robot. IEEE Robot. Autom. Lett. 7 (4), 9881–9888.
Zhang, K., Xu, J., Chen, H., Zhao, J., Chen, K., 2018. Jamming analysis and force control for flexible dual peg-in-hole assembly. IEEE Trans. Ind. Electron. 66 (3), 1930–1939.
Zhang, H.T., Xu, H., Xu, B., Wu, Y., Huang, J., Han, Q.L., 2022. Adaptive learning-based distributed control of cooperative robot arm manipulation for unknown objects. IEEE Trans. Syst. Man, Cybern.: Syst. 53 (2), 1298–1307.
Zribi, M., Ahmad, S., 1992. Adaptive control for multiple cooperative robot arms. In: [1992] Proceedings of the 31st IEEE Conference on Decision and Control. IEEE, pp. 1392–1398.